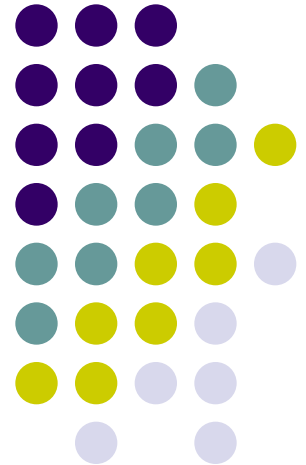
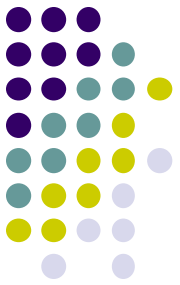


Natural Language Processing

CPSC 488

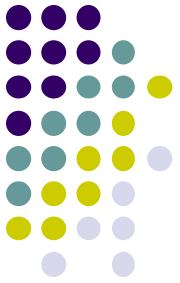
Basics of Machine Learning and Artificial Neural Networks





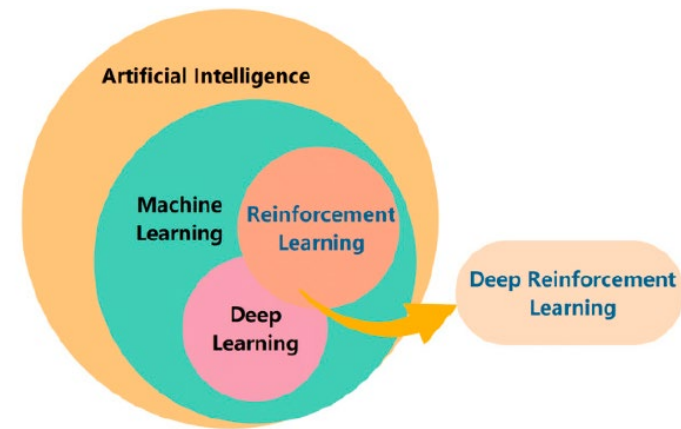
Lecture Overview

- **Concepts of Machine Learning (ML) and ML workflows**
- **Basics of probability theories**
 - Probability distribution, Bayes rule
 - Maximum Likelihood Estimate
 - Uncertainty and Entropy
- **Supervised Learning**
 - Linear regression using Least Squares and Gradient Descent methods
 - Classification using Naïve Bayes and Logistic Regression
- **Artificial Neural Networks**
 - Perceptron and Multilayer Perceptron (MLP)
 - Deep learning architectures



Machine Learning

Machine Learning (ML)



● Definitions

- “A field of study that gives computers the **ability to learn** without being explicitly programmed.” (Samuel, **1959**)
- “Any change in a system that allows it to **perform better** the second time on **repetition** of the **same task** or on **another task** drawn from the same population” (Simon, **1983**)
- “A computer program that learns from **experience E** with respect to some **class of tasks T** and **performance measure P**. If its performance at tasks in T, is measured by P, it **improves** with experience E.” (Tom Mitchell, **1997**)
- **A study of methods to develop a system that can learn**
 - **AI** already **outperforms** humans in many areas and intelligent abilities.
 - The machine learning is the core element of almost all AI systems.

Key Components in Machine Learning

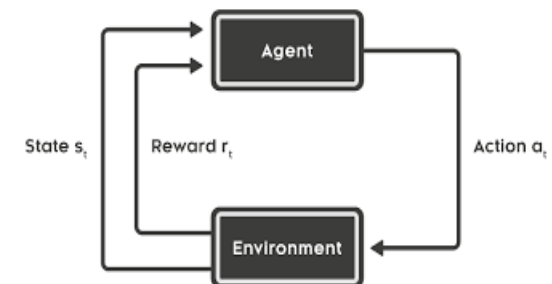
<i>Inflation</i>	<i>Sales</i>	<i>GDP</i>
3.0	100	0.50%
3.4	250	2.00%
4.0	275	2.25%
7.7	200	1.50%
6.5	300	2.50%

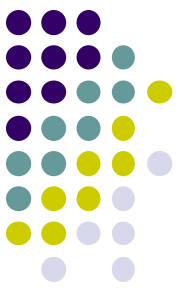
- **Learning goal** (that determines learning tasks) and **algorithm**
 - Finding patterns, relationships, functions, or policies (for an optimal sequence of actions)
 - A learning algorithm is selected based on the learning goal.
- **Input data or experience**
 - **Data** with or without **label**
 - Data with **label** (class, target) is called “**training data**”
 - **Experience** from an environment
- **Output:** Knowledge in different forms
 - **Patterns** (e.g., repeated data, forms, or shapes)
 - **(Concepts) classes** (e.g., Yes/No, high/middle/low)
 - **Clusters** (e.g., a group of data with similar properties)
 - **Rules** (e.g., If credit score > 800, the loan application is approved)
 - **Functions** (e.g., $f(x) = x^2 + 2x + 3$)
 - **Policies of actions** (e.g., making decision or a series of actions)

Image data



Agent's experience





Types of Data for Machine Learning

Dataset

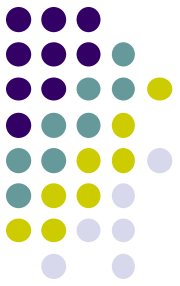
CREDIT HISTORY	DEBT	COLLATERAL	INCOME
bad	high	none	\$0 to \$15k
unknown	high	none	\$15 to \$35k
unknown	low	none	\$15 to \$35k
unknown	low	none	\$0 to \$15k
unknown	low	none	over \$35k
unknown	low	adequate	over \$35k
bad	low	none	\$0 to \$15k
bad	low	adequate	over \$35k
good	low	none	over \$35k
good	high	adequate	over \$35k
good	high	none	\$0 to \$15k
good	high	none	\$15 to \$35k
good	high	none	over \$35k
bad	high	none	\$15 to \$35k

Label Training dataset

RISK	CREDIT HISTORY	DEBT	COLLATERAL	INCOME
high	bad	high	none	\$0 to \$15k
high	unknown	high	none	\$15 to \$35k
moderate	unknown	low	none	\$15 to \$35k
high	unknown	low	none	\$0 to \$15k
low	unknown	low	none	over \$35k
low	unknown	low	adequate	over \$35k
high	bad	low	none	\$0 to \$15k
moderate	bad	low	adequate	over \$35k
low	good	low	none	over \$35k
low	good	high	adequate	over \$35k
high	good	high	none	\$0 to \$15k
moderate	good	high	none	\$15 to \$35k
low	good	high	none	over \$35k
high	bad	high	none	\$15 to \$35k

- **Training datasets**

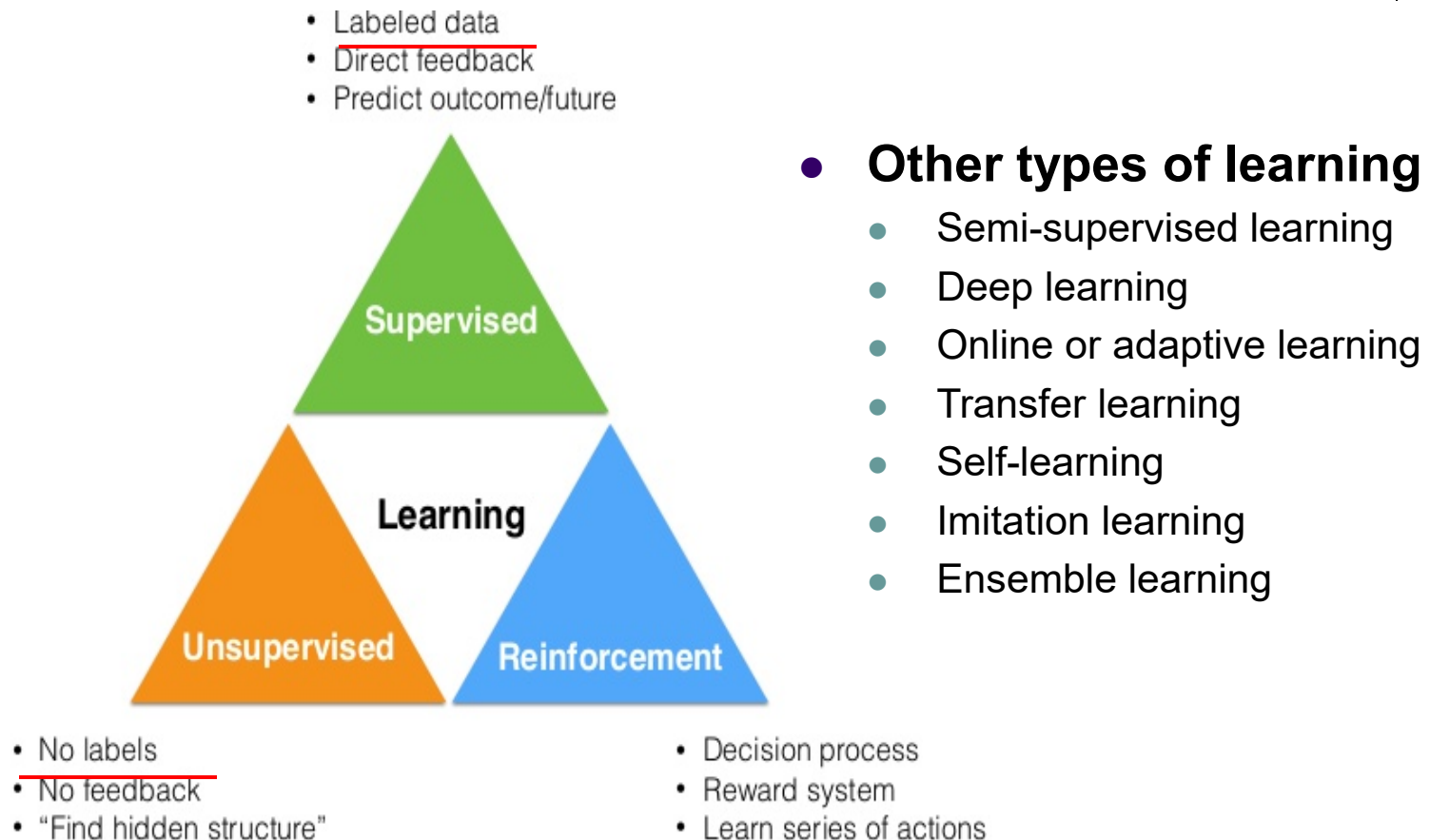
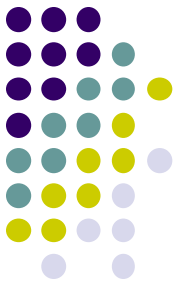
- Datasets collected and categorized based on expert knowledge
- Why are training datasets useful in machine learning?



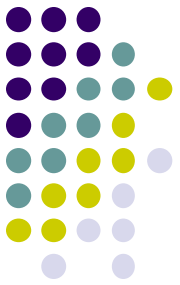
Datasets Repositories for ML *Studies*

- UCI data repository
- Government and public data portals
 - Data.gov (U.S. government data)
 - EU Open Data Portal
- National data platform (<https://nationaldatapatform.org/>)
- Kaggle datasets
- openML
- Google dataset search
- AWS public datasets
- Microsoft research datasets
- Most private datasets are not open

Taxonomy of Machine Learning



Required Background for Ideal ML or AI Engineers



- **Computer Science**

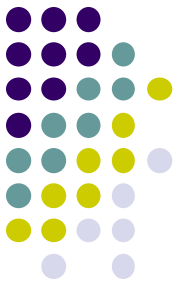
- Data structures, algorithms, databases, data engineering, high-performance computing, computational complexity theory
- Artificial Intelligence (AI), machine learning, natural language processing

- **Mathematics:**

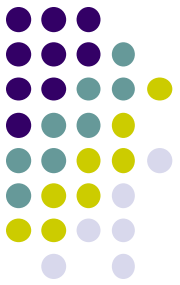
- **Linear algebra** (vectors and matrix, eigenvalues and eigenvectors, matrix factorization, orthogonality, principal component analysis, etc.)
- **Statistics and probability theory** (descriptive statistics, sampling, hypothesis testing, regression analysis, probability distributions, conditional probability, Bayes theorem, etc.)
- **Calculus** (differential and integral calculus, partial derivatives, finding maxima and minima of a function, sigmoid and logit functions, etc.)
- **Other math subjects**
 - Discrete math, Information Theory, numerical analysis, etc.

- **Other related disciplines**

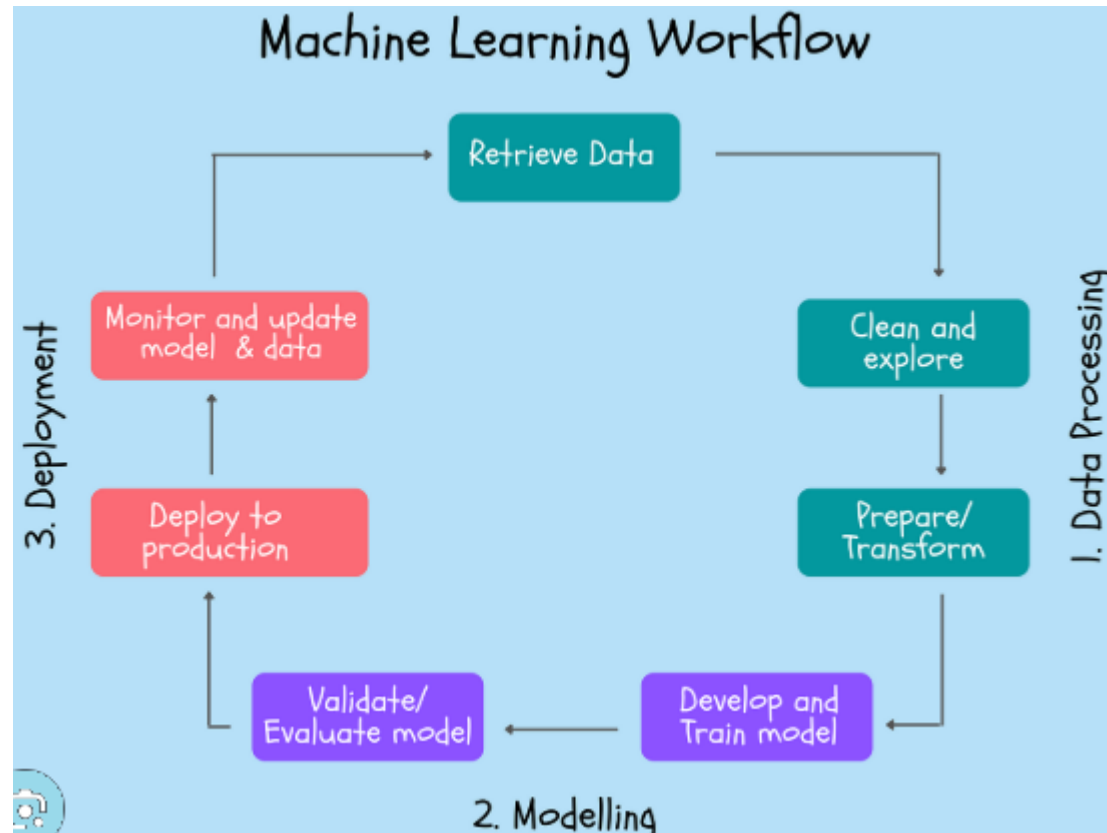
- Decision Science (reinforcement learning, game theory)
- Psychology and Neurobiology (cognitive process of learning)
- **Philosophy** (logics, reasoning, ethics)



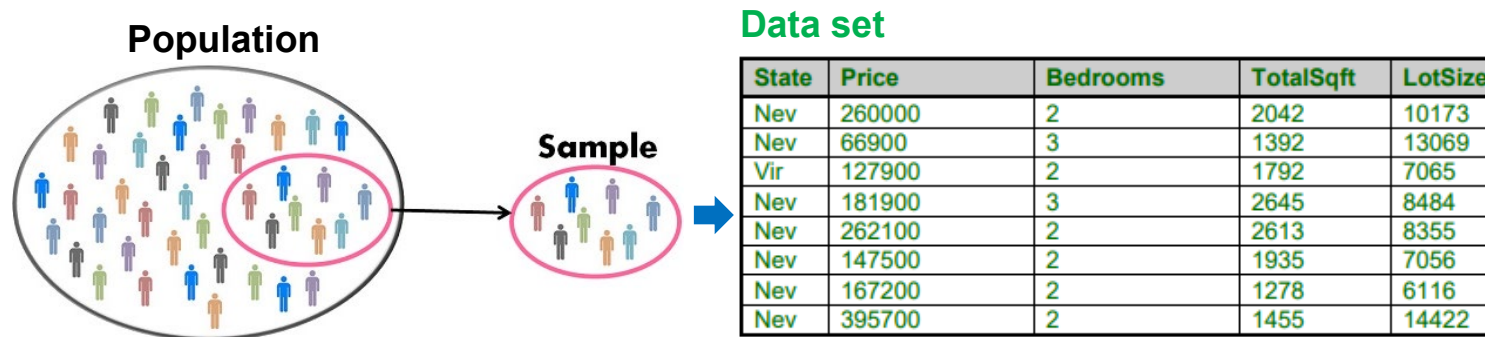
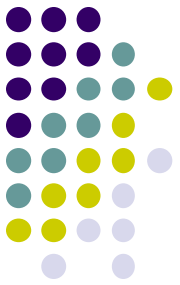
Data Preparation and Preprocessing



Machine Learning Workflow



How to Collect a Data set



- **A population and sample**

- A population is the complete collection of all elements to be studied
- A sample is a sub-collection of members selected from a population.

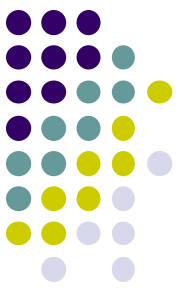
- **Sampling**

- **With replacement**, a member of the population can be chosen more than once.
- **Without replacement**, a member of the population can be chosen only once.
- Sampling error
 - The discrepancy between a sample statistic and its population parameter.
 - **A sample data** must be collected in an **appropriate way** (random and large enough).
- **Misuse of sample**
 - “There are three kinds of lies: lies, damned lies, and **statistics**.” (B. Disraeli)
 - Numbers don’t lie but people can.

Data Engineering



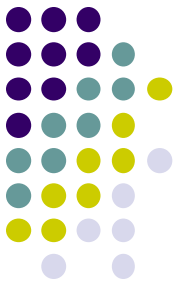
- **Common tasks**
 - Gathering and Cleaning
 - Combining data from multiple data sources (e.g., files, databases, data warehouse)
 - Handling invalid data (NaN, zeros, missing values)
 - Scaling and normalization
 - Converting data on a similar scale.
 - The values of dependent variables are not generally changed except for a few case. Without scaling, gradient descent search can take longer or be lost.
 - Data transformation
 - Discretization of continuous to discrete values, transformation of symbolic values to numerical values, and applying Kernel functions.
 - Sampling
 - Selecting a subset of the data when getting a population data is difficult and expensive
 - Feature selection
 - Features with too many invalid values ($> \lambda$) can be removed.
 - Identifying the significant variables or removing redundant or irrelevant variables for dimensionality reduction.
- **The goal is to prepare the right form of data for the ML algorithms.**



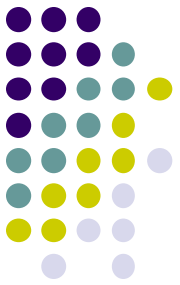
Data Preprocessing

- **Scaling** (converting feature values on a similar scale)
 - Min-max scaling $z = \frac{x - \min(x)}{\max(x) - \min(x)}$
 - Converting values to $[0, 1]$ or $[-1, 1]$ (by dividing them by the largest/smallest value)
- **Normalization** (changing the shape or distribution of the data)
 - Normalization (z score): $z = \frac{x - \bar{x}}{s}$ where s is standard deviation.
 - Mean normalization: $z = \frac{x - \text{mean}(x)}{\max(x) - \min(x)}$
- **Transformation**
 - Discretization of continuous values or symbolic data to numerical values
 - Log transformation $\log(x)$ to address asymmetrical data to reduce variability
 - Apply Kernel functions $K(x, z) \rightarrow \Phi(X) \cdot \Phi(Z)$
 - Unit vector transformation (to normalize the vector)
 - Box Cox transformation (to eliminate heteroskedasticity)
 - Etc.

Dimensionality Reduction

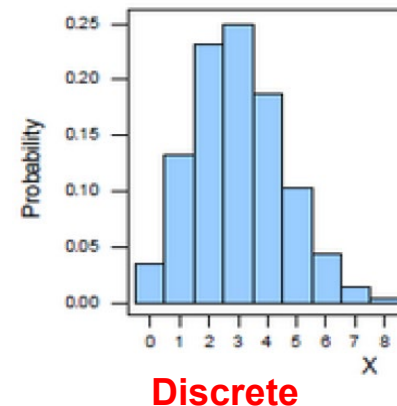


- **Curse of dimensionality** with high-dimensional data (>100 features)
 - With a fixed number of training examples, the predictive power first increases as number of dimensions is increased but then decreases (called “**peaking**” or “**hughes**” phenomenon).
 - Causes **critical issues** in sampling, distance functions, anomaly detection, optimization, analysis, etc.
- **Dimensionality reduction**
 - Modeling the reduced dimension for more efficient, accurate, and simpler model.
 - **Some approaches**
 - Data preprocessing
 - Backward feature elimination, forward feature construction
 - Regularization methods: Ridge, LASSO
 - Principle Component Analysis (PCA)
 - Linear or Generalized Discriminant Analysis
 - Clustering



Basics of Probability Theories

Random Variables and Probabilities



- **Why probability in machine learning?**

- The world is a very uncertain place!
- One of **the best way to estimate a future event** and **quite successful**.

- **Random variables and probability axioms**

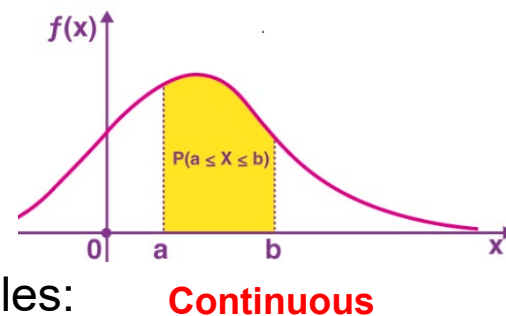
- A random variable X represents uncertain outcomes under a domain.
- $P(X = x)$, $P(x)$, or $Pr(x)$ is the probability that the value or event x is observed.

- Probability mass function $p(x)$ for **discrete** variables:

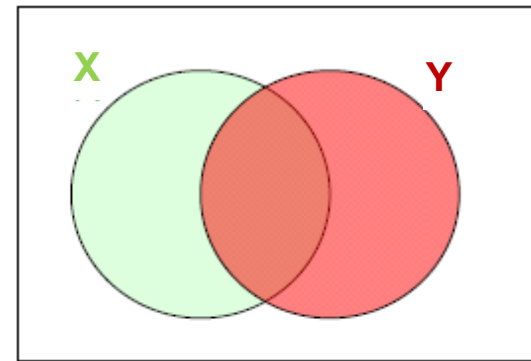
$$0 \leq p(x) \leq 1, \sum p(x) = 1$$

- Probability density function $f(x)$ for **continuous** variables:

$$\int f(x)dx = 1, \int_b^a f(x)dx \geq 0$$

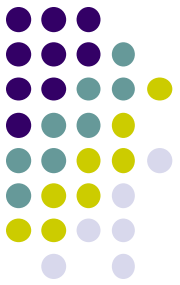


Fundamental Probability Rules



- **Addition:** $P(X \cup Y) = P(X) + P(Y) - P(X \cap Y)$
 - If events X and Y are **independent** (disjoint), $P(X \cup Y) = P(X) + P(Y)$
- **Conditional probability:** $P(X|Y) = \frac{P(X, Y)}{P(Y)}$
 - If events X and Y are **independent**, $P(Y|X) = P(Y)$ or $P(X|Y) = P(X)$
- **Joint probability:** $P(X, Y) = P(X)P(Y|X) = P(Y)P(X|Y)$
 - Can be derived from conditional probability
 - If events X and Y are **independent**, $P(X, Y) = P(X)P(Y)$
- **Chain rule** for any sequence of events $X_1, X_2, \dots, X_n$
 - $P(X_1, X_2, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2) \dots P(X_n|X_1, X_2, \dots, X_{n-1})$ or
 - $P(X_1, X_2, \dots, X_n) = P(X_1|X_2, X_3, \dots, X_n)P(X_2|X_3, X_4, \dots, X_n) \dots P(X_{n-1}|X_n)P(X_n)$
 - $P(X_1, X_2, \dots, X_n) = \prod_{k=1}^n P(X_k|X_1, X_2, \dots, X_{k-1})$

Probability, Likelihood, and Odds



- **Probability:** $P(X = x)$

- A measure of an outcome x of X with the **known model parameter** θ , computing the **ratio** of occurrence of an outcome.
 - E.g., assuming $\theta = 0.5$ (fair coin), what is the probability of getting 3 heads in 10 tosses?
 - **The probability axioms hold**, $0 \leq p(x) \leq 1$, $\sum p(x) = 1$

- **Likelihood** (first coined by **Fisher**): $L(\theta|X)$ or $P(X|\theta)$

- A measure of how well a **model parameter** θ explains the observed data or the probability of observing the data under the **unknown model parameter** θ .
 - $P(X|\theta)$ is **quantitative measure**, not a probability. **The probability axioms do not hold.**

- **Why is the likelihood thinking useful in machine learning?**

- We can ask a question in terms of the **maximum likelihood estimate** like “**For which model parameter** does the observed data have the biggest probability?”

- **Odds**

- The **ratio** between the probability **in favor** (p) and the probability **against it** ($1-p$)
odds = $\frac{p}{1-p}$ or $p : (1-p)$
 - A **quantitative measure** in $[0, \infty]$. **The probability axioms do not hold.**

Probability Distribution for Discrete Variables



- **Probability distribution** for random variables T, W

- Probabilities in a table, graph, or in a function
- $P(W = \text{rain}) = 0.1, T(\text{cold}) = 0.5$

$P(T)$

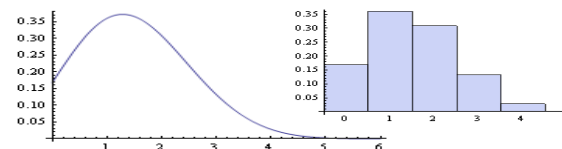
T	P
warm	0.5
cold	0.5

$P(W)$

W	P
sun	0.6
rain	0.1
fog	0.3
meteor	0.0

- **Joint distributions** for variables T, W

- $P(t_1, \dots, t_n)$, e.g., $P(\text{hot}, \text{sun}) = 0.4$



Probability distribution in graphs for continuous and discrete variables

- **Marginal distributions** $P(T)$ and $P(W)$ from $P(T, W)$

- **Marginalizing** (summing out) rows, eliminating some variables

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

$$P(t) = \sum_w P(t, w)$$

$$P(w) = \sum_t P(t, w)$$

T	P
hot	0.5
cold	0.5

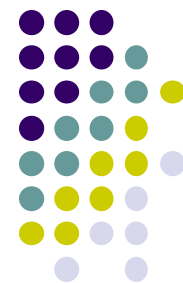
W	P
sun	0.6
rain	0.4

T	W	P
hot	sun	0.4
hot	rain	0.1
cold	sun	0.2
cold	rain	0.3

- **Expected value** (Mean), Variance, and Standard deviation

- **Mean:** $\mu = \sum xP(x) = E(X)$, **variance:** $\sigma^2 = \sum (x - \mu)^2 P(x)$, **standard deviation:** $\sqrt{\sigma^2}$

Bernoulli Distribution (for discrete variables)



- **Bernoulli distribution**

- Models a random variable for a single trial with binary outcomes “success” or “failure” (1/0, yes/no, true/false)

- **Probability Mass Function (PMF):**

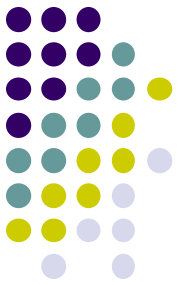
$$P(X = x) = p^x(1 - p)^{1-x} \text{ where } x \in \{0,1\}$$

- Probability of success: $P(X = 1) = p$ if $x = 1$
- Probability of failure: $P(X = 0) = 1 - p$ if $x = 0$
- **Example**
 - The coin flipping event X follows the Bernoulli distribution.

- **Applications**

- Modeling binary events or binary classification
- Building blocks for other distributions such as Binomial distribution, which models the number of successes in multiple Bernoulli trials.

Binomial Distributions (for discrete variables)



- **Binomial distribution**

- Models the probability of obtaining **k** number of successes in **n** number of independent trials, where each trial has binary outcome.
 - Bernoulli distribution is a special case for only single trial ($n=1$, $k=\{0,1\}$).

- **Probability Mass Function**

- $P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}$ where $\binom{n}{k} = \frac{n!}{k!(n-k)!}$
 - X is the random variable representing the number of successes.
 - $\binom{n}{k}$ is the binomial coefficient, representing the number of ways to choose k successes out of n trials.
 - p is the probability of success for each trial. Each trial has the same probability.

- **Example**

- Consider the fair coin are flipped 10 times. The probability of getting 4 heads is
$$P(X = 4) = \binom{10}{4} 0.5^4 (1 - 0.5)^{10-4}$$

- **Applications**

- Modeling the number of successes in experiments
- Binary classification for a dataset with binary label values

Multinomial Distributions (for discrete variables)



- **Multinomial distribution**

- A generalization of the Binomial distribution to multiple outcomes.

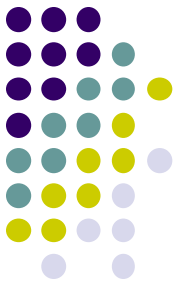
- **Probability Mass Function**

- $$P(X_1 = x_1, X_2 = x_2, \dots, X_k = x_k) = \frac{n!}{\prod_i^k x_i!} \prod_i^k p_i^{x_i!}$$
 - Each of the k outcomes (classes) has a probability, $p_1, p_2, \dots, p_k$ where $\sum_i^k p_i = 1$
 - n is the total count of outcomes where x_i represents the number of occurrences of each outcome $x_1, x_2, \dots, x_k$ where $\sum_i^k x_i = n$
 - The binomial distribution is a special case of the multinomial when $k=2$
- **Example case**
 - When rolling a die, the probability of seeing 1 twice, 2 once, 3 once, 4 once is:
$$P(X_1 = 2, X_2 = 1, X_3 = 1, X_4 = 1) = \frac{5!}{2!1!1!1!} \cdot p_1^2 p_2^1 p_3^1 p_4^1$$

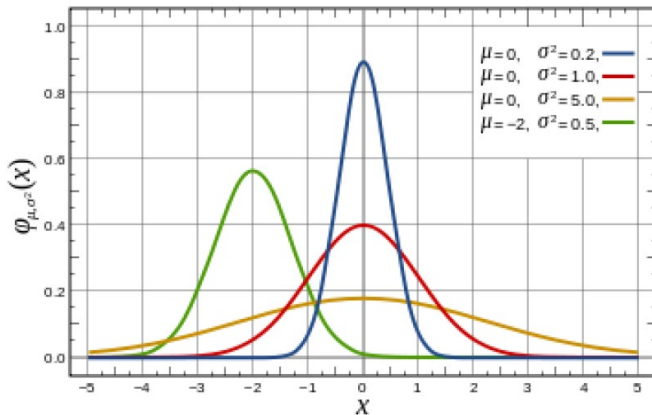
- **Applications**

- Categorical data analysis
- Language modeling

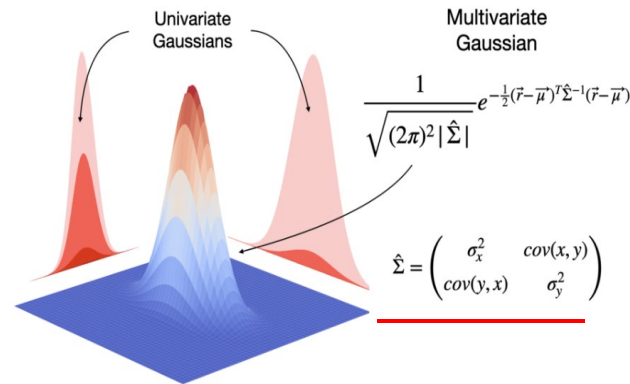
Gaussian (Normal) Distribution (for continuous variables)



Single variable



Multiple variables



The law of large numbers

The central limit theorem

- The **probability density function (PDF)**:

$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}, \text{ standard normal distribution: } \mu = 0 \text{ and } \sigma^2=1$$

- PDF for multivariate Gaussian is shown above figure for multiple variables.
- PDF provides the **likelihood** that the value of a random variable will fall **between** a certain **range** of values.

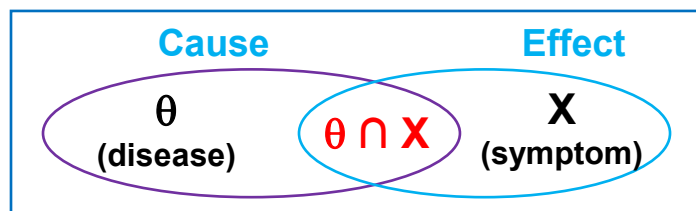
- **Calculating probabilities**

- The **probability** of the outcome is the **area** of the range ($\int_b^a f(x)dx$).

Bayesian Approach and Bayes' Rule



Bayes, 1763



Example case

Given symptoms X (effect, data), what is the probability of having the disease θ , $P(\theta|X)$?

In many cases, given symptoms (effect) X , finding the disease (cause, hypothesis) θ , computing $P(\theta|X)$ is difficult.

Often, it is easier to compute the probability of disease, $P(\theta)$ and the probability of observing the symptoms X given the disease θ , $P(X|\theta)$.

Deriving Bayes' rule

- From $P(X|\theta) = \frac{P(X,\theta)}{P(\theta)}$, $P(X,\theta) = P(X|\theta)P(\theta)$

$$P(\theta|X) = \frac{P(\theta,X)}{P(X)} = \frac{P(X,\theta)}{P(X)} = \frac{P(X|\theta)P(\theta)}{P(X)}$$

- The **Bayes rule**: $P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$
 - θ is a hypothesis or parameter to find.
 - X is the evidences or observed data.

Why is Bayes' rule useful?

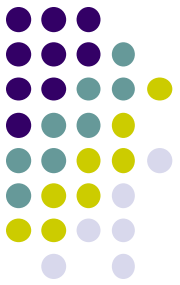
- When one conditional is tricky to compute but the other conditional can be simple,
- Bayes' rule lets us build one conditional from its reverse.
 - Simple idea but amazingly powerful!**
 - Foundation for **many practical systems**

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

$$\textit{Posterior} = \frac{\textit{Likelihood} * \textit{Prior}}{\textit{Marginal}}$$

Elements of Bayes' Rule

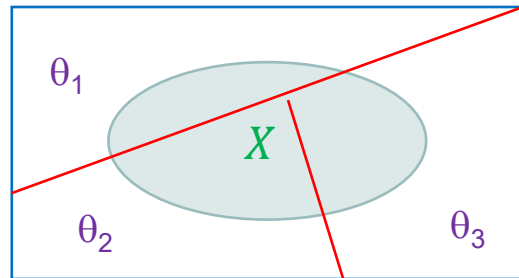
- **Posterior probability $P(\theta|X)$** : P(hypothesis | evidence)
 - Given the evidence, the probability of the hypothesis, $P(\text{flu} \mid \text{"fever"})$
- **Likelihood $P(X|\theta)$** : P(evidence | hypothesis)
 - Given that the hypothesis is true, the likelihood of the evidence.
 - If the evidence is consistent with hypothesis, it will increase the posterior probability, $P(\text{"fever"} \mid \text{flu})$.
- **Prior probability $P(\theta)$** : P(hypothesis)
 - The probability of the hypothesis prior to any evidence, based on **prior knowledge**, $P(\text{flu})$.
- **Marginal likelihood $P(X)$** : P(evidence)
 - The **sum** of all probabilities related to the evidence fever from all the other hypotheses.
 - New evidence will update the posterior probability.



Marginal Likelihood in Bayes' Rule

- **Marginal likelihood**

- Assume that the entire sample space and event X within it, are **partitioned** by the set of **disjoint hypotheses** $\theta = \cup \theta_i$.

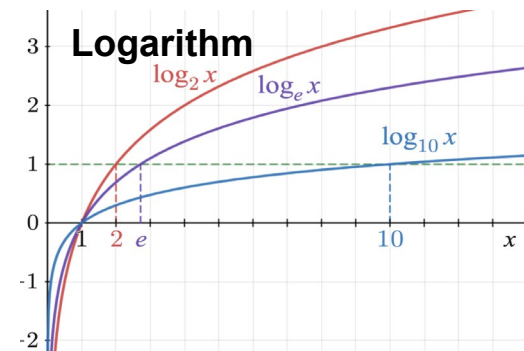


- $P(X) = P(\cup_i P(\theta_i, X)) = P(\theta_1, X) + P(\theta_2, X) + \dots + P(\theta_i, X)$
- By the *total probability theorem* $P(X) = \sum_i P(X|\theta_i)P(\theta_i)$
 - since $P(\theta, X) = P(\theta)P(X|\theta)$ from $P(X|\theta) = \frac{P(\theta, X)}{P(\theta)}$

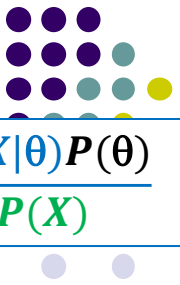
- **Bayes' rule:** $P(\theta_i|X) = \frac{P(X|\theta_i)P(\theta_i)}{P(X)} = \frac{P(X|\theta_i)P(\theta_i)}{\sum_j P(X|\theta_j)P(\theta_j)}$

- Note: i is one of j and $\sum_j P(X|\theta_j)P(\theta_j)$ is a **normalization factor** to return a value **[0,1]**

Maximum Likelihood Estimate (MLE)



- **Likelihood:** $L(\theta|X)$ or $P(X|\theta)$
 - How well the model parameters θ explain the observed data X , giving the probability of the observed data X given θ .
 - It is a quantitative measure (instead of probability) of how well the parameter values explain the observed data.
- **MLE:** $\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} P(X|\theta)$
 - Seeks to find θ that maximize the likelihood $P(X|\theta)$.
 - Taking log: $\hat{\theta}_{MLE} = \operatorname{argmax}_{\theta} \log P(X|\theta)$
- **Why taking log?**
 - Multiplying numbers $[0,1]$ many times results in smaller and possibly go to infinity (**underflow**)
 - The logarithm function **monotonically increases**.
 - **Maximizing a likelihood $\equiv$ maximizing the log likelihood.**
 - Taking **derivative** of **log** (with multiplications) is **easier as it becomes additions!**

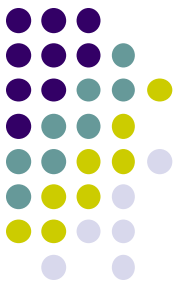


Prior Knowledge and More Data

$$P(\theta|X) = \frac{P(X|\theta)P(\theta)}{P(X)}$$

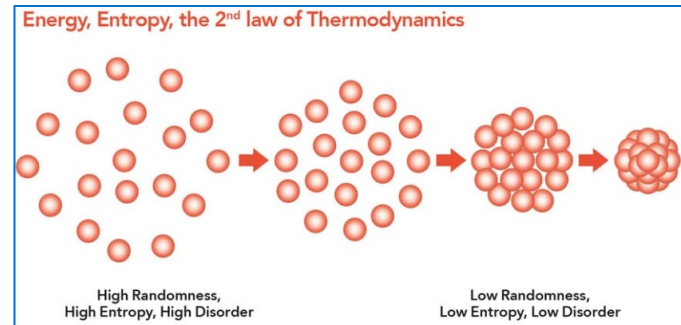
- **Strength of MLE** $\theta_{MLE} = \operatorname{argmax}_{\theta} P(X|\theta) = \operatorname{argmax}_{\theta} \prod_i P(x_i|\theta)$
 - MLE gives the explanation of the data you observed.
 - If the model distribution is correct with enough data, MLE finds the true parameter θ .
- **Problem of MLE**
 - MLE favors complex models that may **overfit** if the data size is small.
 - e.g., suppose you observe all five heads. What is $\hat{\theta}_{MLE}$?
- **The influences of prior knowledge and more data**
 - What if I have **prior knowledge** $P(\theta)$?
 - Better to incorporate prior knowledge
 - What if **more data (X, evidences)** are added later?
 - **Marginal likelihood** becomes more concentrated around θ that **better explain the data** – the **likelihood term** is more influential and **prior** becomes less important
 - **Asymptotically approach to true model θ .**
 - **Even better idea:**
 - Obtain a **distribution** over all parameter values instead of estimating a **single θ .**

Maximum a Posteriori (MAP)



- Estimating θ using the posterior probability $P(\theta|X)$
- $\hat{\theta}_{MAP} = \operatorname{argmax}_{\theta} P(\theta|X) = \operatorname{argmax}_{\theta} \frac{P(X|\theta)P(\theta)}{P(X)} \propto \operatorname{argmax}_{\theta} P(X|\theta)P(\theta)$
 - Find θ (point estimation) that **maximize** the posterior $P(\theta|X)$
 - maximizing $P(\theta|X) \propto$ maximizing $P(X|\theta)P(\theta)$
 - **How to estimate the prior probability $P(\theta)$?**
 - For **binary classification (Binomial)**, the most common form of prior distribution $P(\theta)$ is a **Beta distribution**:
$$P(\theta) = \operatorname{Beta}(\beta_0, \beta_1) = \frac{\theta^{\beta_1-1}(1-\theta)^{\beta_0-1}}{B(\beta_0, \beta_1)}$$
where β_0, β_1 are parameters.
 - Computing the **posterior $P(\theta|X)$** or **joint probability distribution $P(\theta, X)$** , is **generative modeling**.
- **MLE vs MAP**
 - Both can be used for estimating model parameters.
 - **MAP** seeks to find the model parameters θ that maximize the posterior probability $P(\theta|X)$ given the observed data X using likelihood $P(X|\theta)$ and prior $P(\theta)$ while **MLE** seeks to find θ that maximizes the likelihood of observing X , $P(X|\theta)$.

Uncertainty and Entropy



- **Entropy in physics**

- If the particles inside a system have many possible positions to move around, then, the system has high entropy.
 - e.g., water in solid state (low entropy), water in gas state (high entropy)

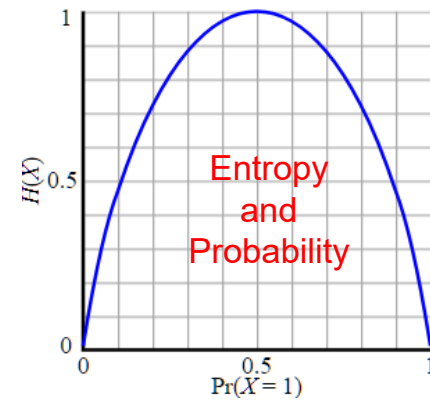
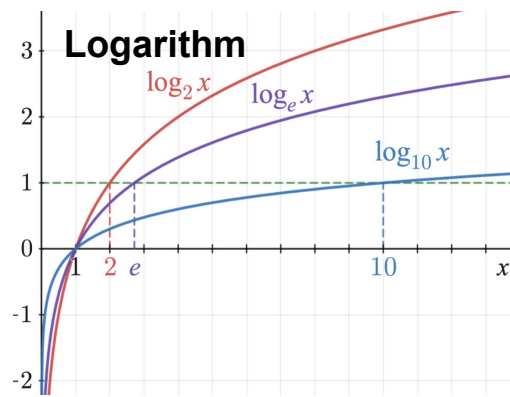
- **Entropy in information theory** (Claude Shannon, 1916-2001)

- **Entropy** is a measure of the **uncertainty** associated with a **random variable**.
 - The **average amount of information** one should expect (or **surprise**) when observing a value drawn from a **probability distribution**
 - How much information is acquired from the observation of a random event?
 - That depends on how much information the event carries.
 - The more information it carries, the higher entropy.
- Higher entropy indicates greater uncertainty or randomness.
- Lower entropy indicates less uncertainty, more predictability.

- **Entropy and knowledge**

- The more we (model) know, the less uncertain the event (variable) is.

Shannon's Entropy



- **Entropy $H(X)$**

- Entropy of a **discrete random variable** X with probability mass function $P(X)$ is given:

$$H(X) = \sum_{i=1}^N P(x_i) I(x_i) \text{ where Information content } I(x_i) = -\log P(x_i)$$

$$H(X) = -\sum_{i=1}^N P(x_i) \log P(x_i) \text{ Why negative log?}$$

- **Interpretation:**

- **Shannon's entropy** is measured in **discrete binary bits** (the log base 2).
- The **expected** (average) number of **bits needed to encode** an outcome from the random variable X .
- The more **unpredictable** the outcome is, **more bits** needed.

- **Properties of $H(X)$: $0 \leq H(X) \leq \log(1/N)$**

- Higher entropy when outcomes are smoothly spread.
 - **The maximum entropy** is **$\log(1/N)$** when every outcome has equal chance, $p(x) = 1/N$.
- Lower entropy when outcomes are concentrated.
 - **The minimum entropy** is **zero** when $p(x) = 1$, $\log(1) = 0$ or $p(x) \approx 0$ ($\log(0)$ is undefined).

Examples: Entropy



• Fair coin tossing

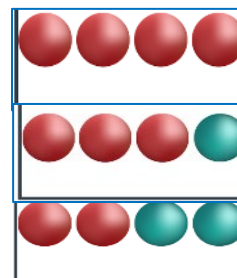
- For a fair coin tossing, what is the entropy for the coin tossing event X ?
 - The possible values of X : h(head) or t(tail)
 - $p(h) = 0.5$. $p(t) = 0.5$. $H(X) = -\sum_{i=1}^N p(x_i) \log_2 p(x_i)$
 - $H(X) = -p(h)\log_2 p(h) - p(t)\log_2 p(t) = -1/2\log_2(1/2) - 1/2\log_2(1/2) = 1 \text{ bit}$.
 - It means that **one bit** is required to encode the message of the coin tossing.

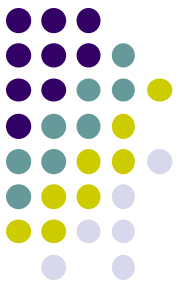
• Rigged coin tossing

- For a rigged coin tossing with **75%** chance of seeing heads, what is the entropy for the coin tossing event X ?
 - Now, we know more about the coin, $p(h)=3/4$ and $p(t)=1/4$.
 - $H(X) = -3/4\log_2(3/4) - 1/4\log_2(1/4) = 0.811 \text{ bits}$.

• Three boxes with colored balls

- What is the entropy of each box?



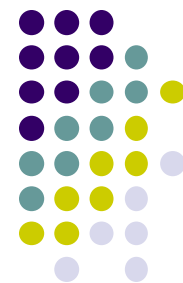


Entropy of a Training Data Set

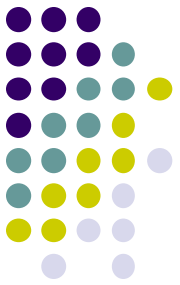
Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

- **Measuring the entropy of a training dataset**
 - What's the entropy of the dataset in terms of a decision on “playing golf”?
 - $P(\text{Yes})=9/14=0.64$, $P(\text{No})=5/14=0.36$, $H(\text{PlayGolf}) = -[0.64*\log 0.64 + 0.36*\log 0.36] \approx 0.94$
 - Entropy can be used to measuring **impurity** of a data set.
 - Pure or homogeneous data set contains **only a single class**. Otherwise, the data set is impure or heterogenous.

Cross-entropy



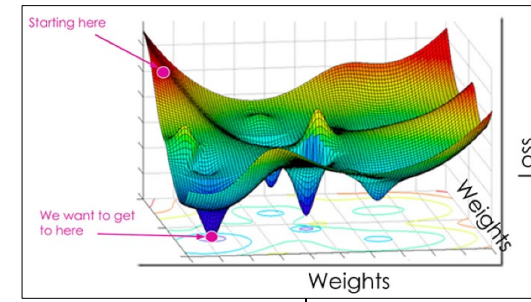
- **Cross-entropy:** $H(P, Q) = -\sum_i P(x_i) \log Q(x_i)$
 - The **difference** (divergence) between **two probability distributions** P and Q for a **given random variable X**
 - If Q is close to P, the cross-entropy is small. If Q **diverges** from P, the cross-entropy is larger.
 - The **average number of bits** needed to encode data coming from a source with distribution P (true distribution) when we use model Q (approximation of the true or predicted distribution).
 - A quantitative measure of the **dissimilarity** between two probability distributions
- **The Kullback-Leibler divergence** (or relative entropy): $D_{KL}(P||Q)$
 - $D_{KL}(P||Q) = \sum_i P(x_i) \log \left(\frac{P(x_i)}{Q(x_i)} \right)$
 - If P and Q are identical, $D_{KL}(P||Q) = 0$.
 - Asymmetric: $D_{KL}(P||Q) \neq D_{KL}(Q||P)$
 - Cross-entropy based on divergence: $H(P, Q) = H(P) + D_{KL}(P||Q)$
 - Minimizing $H(P, Q)$ with respect to Q $\equiv$ minimizing $D_{KL}(P||Q)$



Examples: Cross-entropy

- Entropy and cross-entropy
 - Assume the following probability distributions:
 $P = [0.3, 0.5, 0.2]$
 $Q = [0.4, 0.4, 0.2]$
 - **Entropy:** $H(P) = -\sum_i P(x_i) \log P(x_i)$
 - $H(P) = -[0.3 \log 0.3 + 0.5 \log 0.5 + 0.2 \log 0.2] \approx 1.4855$, $H(Q) \approx ?$
 - $H(P)$ value represents the average number of bits needed to encode an event drawn from the distribution P .
 - **Cross-entropy:** $H(P, Q) = -\sum_i P(x_i) \log Q(x_i)$
 - $H(P, Q) = -[0.3 \log 0.4 + 0.5 \log 0.4 + 0.2 \log 0.2] \approx 1.3219$ bits
 - This value represents the average number of bits needed to encode events from P using the optimal code for Q .
- The Kullback-Leibler divergence
 - $D_{KL}(P||Q) = \sum_i P(x_i) \log \left(\frac{P(x_i)}{Q(x_i)} \right)$
 - $D_{KL}(P||Q) = 0.3 \log \left(\frac{0.3}{0.4} \right) + 0.5 \log \left(\frac{0.5}{0.4} \right) + 0.2 \log \left(\frac{0.2}{0.4} \right) \approx -0.1245 + 0.1609 - 0.2 = -0.1635$

Uses of Cross-entropy



- **Various uses of cross-entropy**

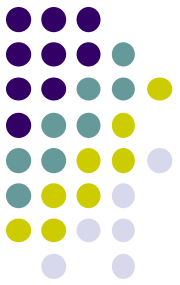
- Measuring the **difference** between **predicted** and **actual values** in **classification**
- How well one probability distribution predicts another in **recommended systems**
- Etc.

- **Cross-entropy in classification**

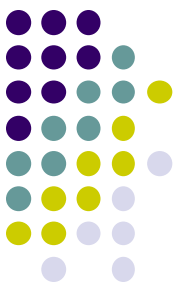
- For **binary class**
 - y is the true label (0 or 1) and $\hat{y}$ is the predicted label $[0, 1]$ calculated in probability
 - For **$y=1$** : $H(y = 1, \hat{y}) = -y \log \hat{y}$ and for **$y=0$** : $H(y = 0, \hat{y}) = -(1 - y) \log(1 - \hat{y})$
 - **Combining both cases**: $H(y, \hat{y}) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$
- For **multi-class**: $H(y, \hat{y}) = -\sum_i y_i \log \hat{y}_i$ (generalization of binary class)

- **Cross-entropy as loss function**

- Find θ that minimizes $H(y, \hat{y})$ to optimize classification models
 - $H(y, \hat{y})$ measures the difference between actual class y (probability distribution) and predicted class $\hat{y}$ (probability distribution) for a given example x .
 - $H(y, \hat{y})$ is minimized when both y and $\hat{y}$ are identical.
 - **Works better than** the **squared sum** as it leads to faster training as well as improved generalization

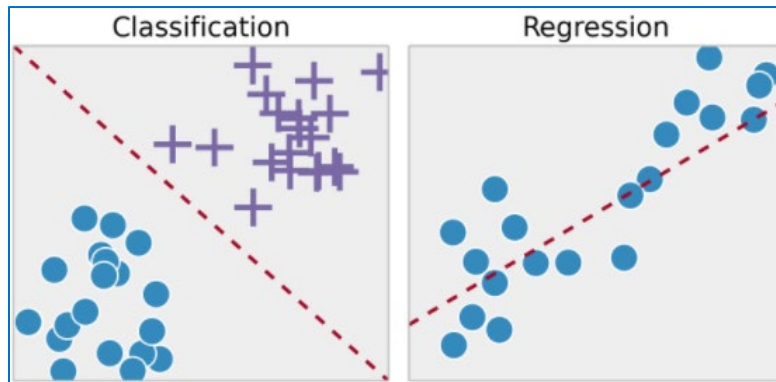


Supervised Learning

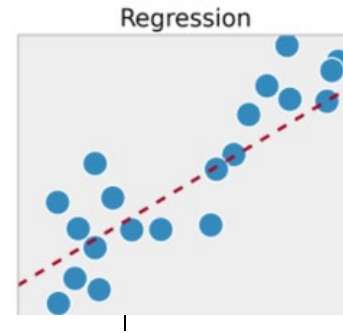


Supervised Learning

- Given a training data set $\{(X_1, \dots, X_k, y_j) | j = 1 \dots N\}$
 - Learn a **function** or **classifier** $f: x \rightarrow y$
 - The process of learning f such that $f(x) \approx y$ for new data points, is called **training**.
 - For new data point x ,
 - If the learned **function** f is used to predict a continuous value, $\hat{y}$, it's called **regression**.
 - If the learned **classifier** f is used to predict a discrete value, $\hat{y}$, it's called **classification**.
- Two major tasks:** Classification vs. regression
 - Classification** is to find the model that **separates** the different groups of data as clearly as possible.
 - Regression** is to find the **function** that **fits** the data the best.

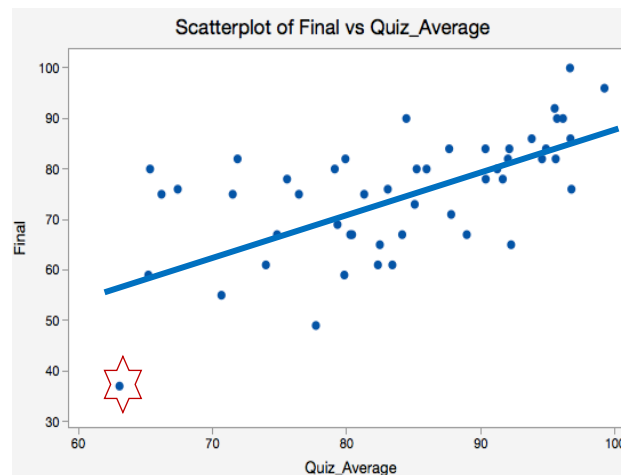


Supervised Learning for Regression



- Given a **training data set** $\{(X_1, \dots, X_k, y_j) | j = 1 \dots N\}$
 - Learn a **function** or **classifier** $f: x \rightarrow y$
 - The process of learning f is called **training**.
 - For a given X (new data):
 - If the learned **function** f is used to predict a continuous value, $\hat{y}$, it's called **regression**.

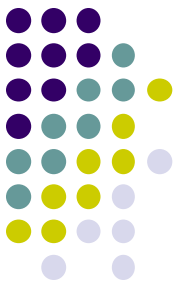
x	y
QuizAvg	Final
64	38
66	59
67	80
68	77
71	58
...	...



The result of
regression modeling

$$f(x) = y = 0.75x + 51$$

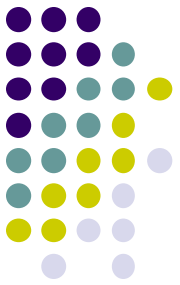
$$W = [51, 0.74]$$



Training Data Set for Regression

Estimate the gas quality (Wobbe Index) based on gas properties. $y = w^T X$

Temperature, T	Pressure, P	Composition			Thermal Conductivity, $k \times 10^3$	Sound Velocity, v	Wobbe Index
		CH ₄	C ₂ H ₆	CO ₂			
° C	psi	%	%	%	W/(m.K)	m/s	MJ/Nm ³
-20	600	94.00	6.00	0.00	27.36	350.86	54.90
-20	600	91.83	5.40	2.77	27.02	343.90	52.16
-20	600	89.66	4.80	5.54	26.68	336.48	49.53
-20	600	87.49	4.20	8.31	26.33	330.29	47.03
-20	600	85.32	3.60	11.08	25.99	324.43	44.62
-20	600	83.15	3.00	13.85	25.64	318.88	42.32
-20	600	80.98	2.40	16.62	25.30	313.60	40.10
-20	600	78.81	1.80	19.39	24.95	307.78	37.97
-20	600	76.64	1.20	22.16	24.61	303.00	35.91
-20	600	74.47	0.60	24.93	24.26	298.44	33.92
-20	600	72.3	0.00	27.7	23.92	294.08	32.00
-20	600	71.52	0.00	28.48	23.80	292.74	31.51
-20	600	73.69	0.60	25.71	24.14	297.04	33.41
-20	600	70.74	0.00	29.26	23.68	291.42	31.03
-20	600	75.86	1.20	22.94	24.49	301.54	35.38
-20	600	72.91	0.60	26.49	24.03	295.66	32.91
-20	600	69.96	0.00	30.04	23.56	290.11	30.55
-20	600	78.03	1.80	20.17	24.83	306.24	37.42
-20	600	75.08	1.20	23.72	24.37	300.09	34.86
-20	600	72.13	0.60	27.27	23.91	294.30	32.41
-20	600	69.18	0.00	30.82	23.45	288.83	30.07
-20	600	80.2	2.40	17.4	25.18	311.98	39.53

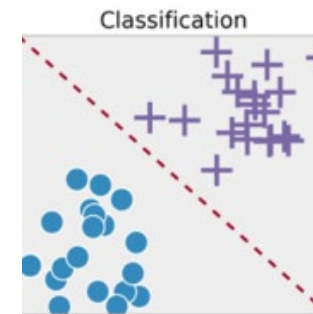


Supervised Learning for Classification

Training dataset

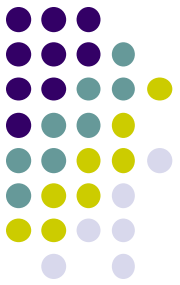
Label: Binary
classification (1/0)

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No



- We need to find a model that correctly classifies the data based on the label values in the training dataset.

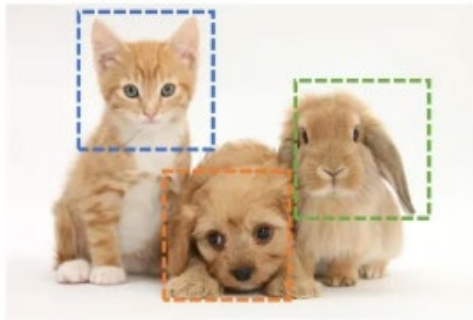
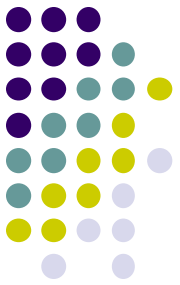
Training Data Set for Classification



Class RISK	CREDIT HISTORY	DEBT	COLLATERAL	INCOME
high	bad	high	none	\$0 to \$15k
high	unknown	high	none	\$15 to \$35k
moderate	unknown	low	none	\$15 to \$35k
high	unknown	low	none	\$0 to \$15k
low	unknown	low	none	over \$35k
low	unknown	low	adequate	over \$35k
high	bad	low	none	\$0 to \$15k
moderate	bad	low	adequate	over \$35k
low	good	low	none	over \$35k
low	good	high	adequate	over \$35k
high	good	high	none	\$0 to \$15k
moderate	good	high	none	\$15 to \$35k
low	good	high	none	over \$35k
high	bad	high	none	\$15 to \$35k

Specially created or collected to find relationships between **explanatory variables** and **response variable** (**class or label**)

Classification Examples



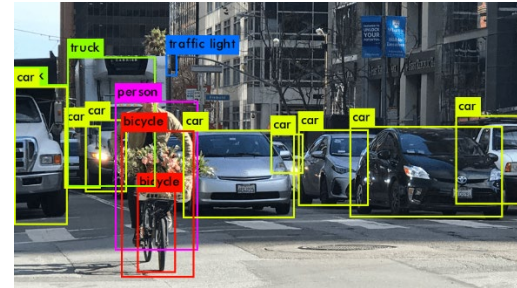
Cat, Dog, Rabbit

Image classification

1		1	1	1	1	1	1	1	1
2		2	2	2	2	2	2	2	2
3		3	3	3	3	3	3	3	3
4		4	4	4	4	4	4	4	4
5		5	5	5	5	5	5	5	5
6		6	6	6	6	6	6	6	6

Numbers in training data

Digit/character recognition or OCR

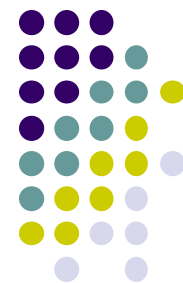


Object detection

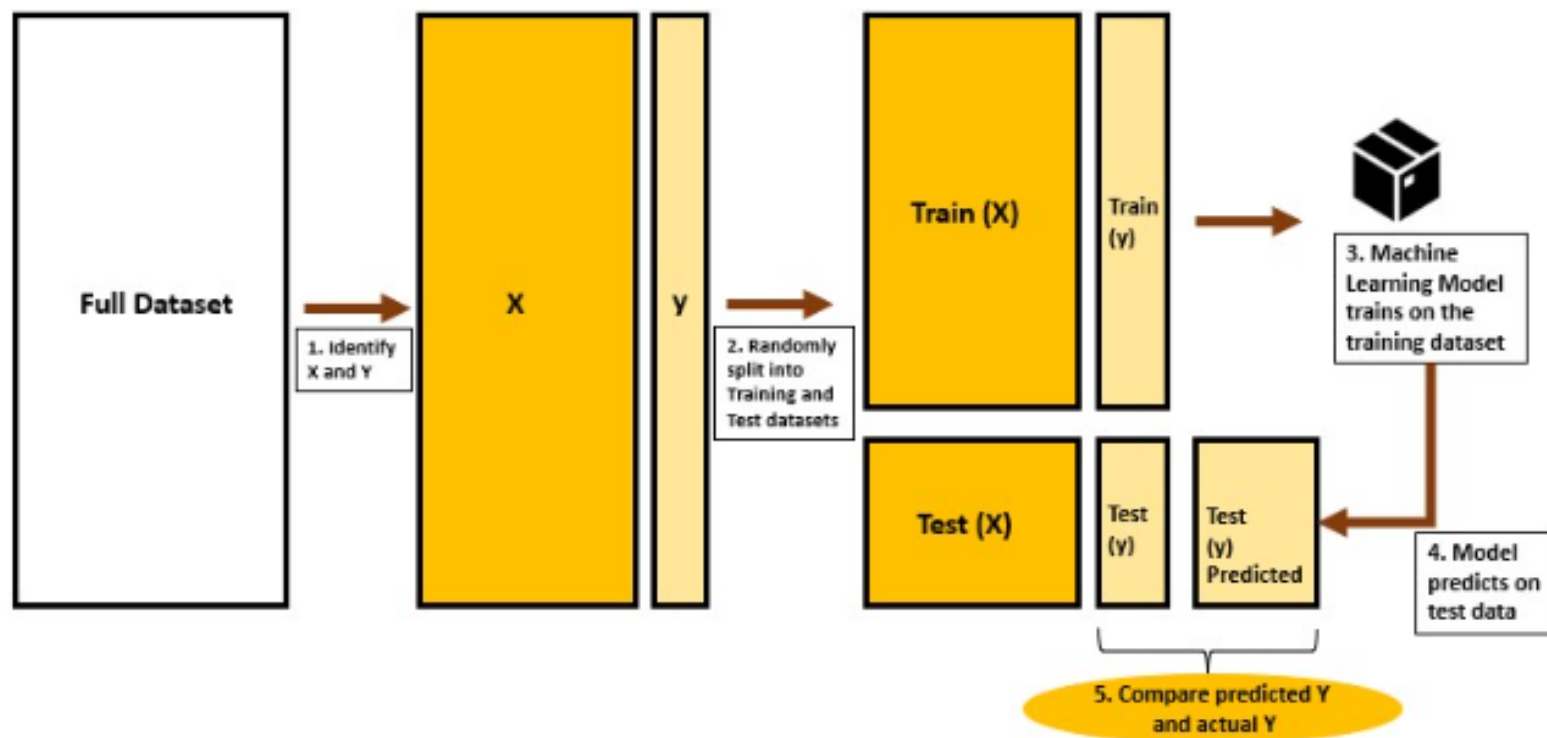


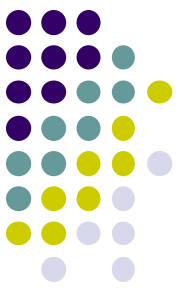
Face recognition

Preparing Datasets for Supervised Learning



- **Training and testing data sets**
 - Training data set for modeling and testing data set for testing the model

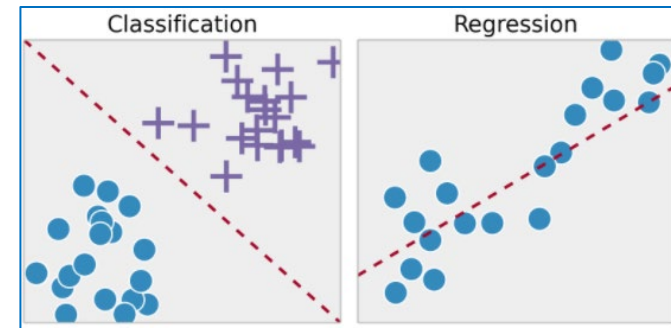




Preparing Data Sets

- **Training data, (verification data), and testing data**
 - Training data for modeling, verification, and testing data for testing the model
- **Methods for splitting the data**
 - **Holdout**
 - A data set is partitioned to training and testing (e.g., 80:20, 2/3:1/3, or 50:50).
 - Random subsampling
 - Repeating Holdout several times and computing the average accuracy.
 - Bootstrap sampling
 - Training data are sampled with **replacement**.
 - The data not included in the bootstrap samples are test data.
 - Cross-validation (CV)
 - All the examples in a data set is used for both training and testing, utilizing as much data as possible.
 - CV is computationally expensive but works well for most algorithms.
 - CV is one of the most popular methods to compare **learning algorithms** and tune **algorithm parameters**.

Supervised Learning Methods

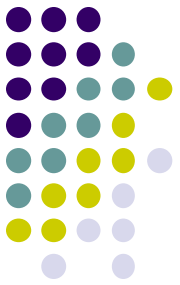


- **Parametric vs Non-parametric**

- **Parametric methods** learn a fixed number of parameters in specific form
 - The least-square method
 - Gradient descent
 - Artificial Neural Networks
 - Etc.
- **Non-parametric methods** learn any number of parameters and do not assume a fixed form.
 - K-Nearest Neighbors (KNN), Decision tree induction, etc.
 - The number or form of the parameters for f is not fixed in advance and can grow with the more data is added.

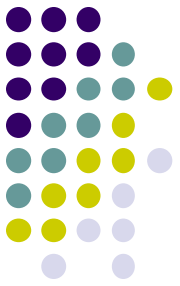
$$y = w^T X$$

- The **supervised learning methods** can be used for **both regression and classification** tasks by making small changes.



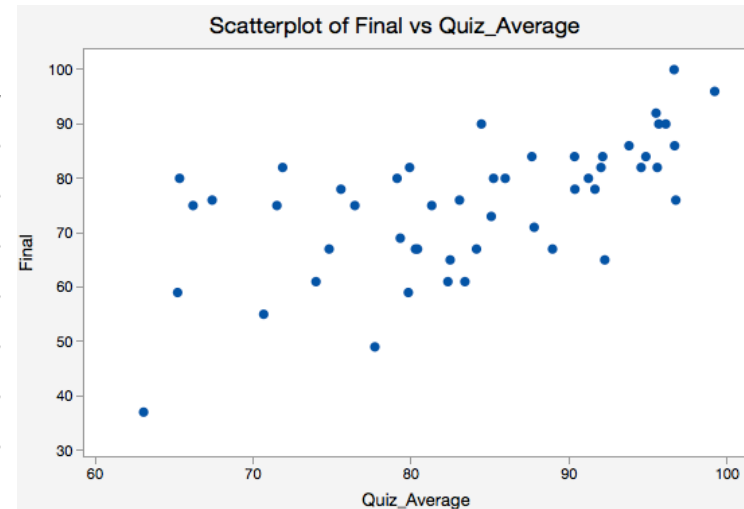
Supervised Learning for Regression

Investigating the Relationships in Numerical Data



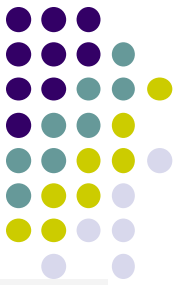
- **Explanatory variable** (**independent** or **predictor** variable) is used to explain the observed outcome in the response variable.
- **Response variable** (**dependent** variable) represents the observed outcome for given values of explanatory variables.

x	y
QuizAvg	Final
64	38
66	59
67	80
68	77
71	58
...	...

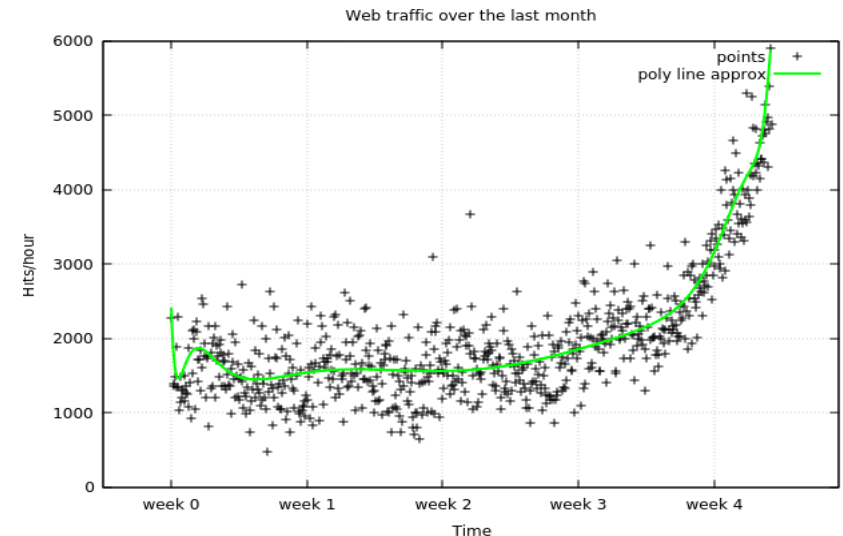
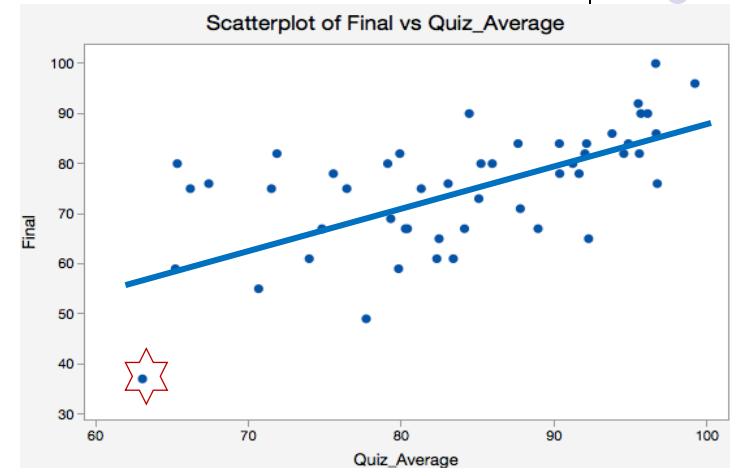


- What are the relationships among variables in a data set?
 - Correlation and its strength
 - Linear/non-linear functional relationships

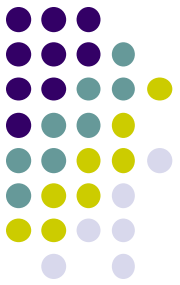
Finding Relationships among Variables



- Explores the way the **variables** **correlate** and **outliers**.
 - Positive/Negative
 - Strong/Weak
 - ✓ Linear/Non-linear
 - ✓ Presence of outliers
- Find a function **f** to **predict** (estimate) **new values**.
 - **Linear or non-linear** (straight or curve line)
 - **Simple or complex** (two or more variables)
- **Terminologies**
 - The **function** that fits the data is called “**model**”.
 - The process of **finding the function** is called “**modeling**” (statisticians) or “**learning**” (computer scientists).
 - Trying to **understand the data** through modeling is called “**regression analysis**”.

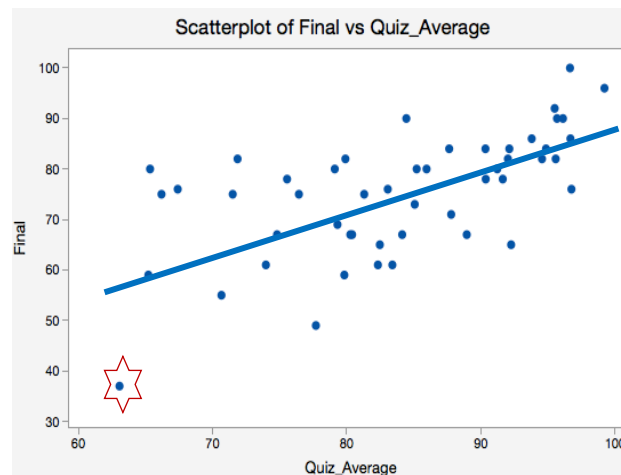


Supervised Learning for Regression



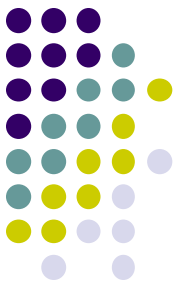
- Given a **training data set** $\{(X_1, \dots, X_k, y_j) | j = 1 \dots N\}$
 - Learn a **function** or **classifier** $f: x \rightarrow y$
 - The process of learning f is called **training**.
 - For a given X (new data):
 - If the learned **function** f is used to predict a continuous value, $\hat{y}$, it's called **regression**.

x	y
QuizAvg	Final
64	38
66	59
67	80
68	77
71	58
...	...



The result of
regression modeling

$$f(x) = y = 0.75x + 51$$



Supervised Learning for Regression

- Given a **training data set** $\{(X_1, \dots, X_k, y_j) | j = 1 \dots N\}$
 - Learn a **function** or **classifier** $f: x \rightarrow y$
 - The process of learning f is called **training**.
 - For a given X (new data):
 - If the learned **function** f is used to predict a continuous value, $\hat{y}$, it's called **regression**.

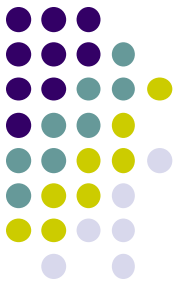
A data set with more than one explanatory variables

<i>Price (K\$) ($Y$)</i>	<i>Size (X_1)</i>	<i>Bedroom (X_2)</i>	<i>Bathroom (X_3)</i>	<i>Built year (X_4)</i>
375 (y_1)	1024 (x_{11})	3 (x_{12})	2 (x_{13})	1978 (x_{14})
425 (y_2)	1329 (x_{21})	3 (x_{22})	5 (x_{23})	1992 (x_{24})
...	...	...	...	...
465 (y_N)	1893 (x_{N1})	4 (x_{N2})	4 (x_{N3})	1980 (x_{N4})

Which function can compute the value of y using the explanatory variables x ?

$$y = w_0 + w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4$$

What are **unknown variables** in this expression?



Regression Model in Vector/Matrix Form

- **Stacking all the equations**

- k number of variables (features)
- N number of equations (rows)

y_1	$=$	w_0	$+$	$w_1 x_{11}$	$+$	$\dots$	$+$	$w_k x_{1k}$
y_2	$=$	w_0	$+$	$w_1 x_{21}$	$+$	$\dots$	$+$	$w_k x_{2k}$
$\dots$		$\dots$		$\dots$		$\dots$		$\dots$
y_N	$=$	w_0	$+$	$w_1 x_{N1}$	$+$	$\dots$	$+$	$w_k x_{Nk}$

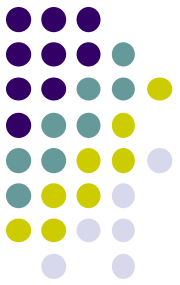
- **The equations in vector form**

- $\begin{bmatrix} y_1 \\ \dots \\ y_N \end{bmatrix} = w_0 \begin{bmatrix} 1 \\ \dots \\ 1 \end{bmatrix} + w_1 \begin{bmatrix} x_{11} \\ \dots \\ x_{N1} \end{bmatrix} + \dots + w_k \begin{bmatrix} x_{1k} \\ \dots \\ x_{Nk} \end{bmatrix}$ or $Y = w_0 1 + w_1 X_1 + \dots + w_k X_k$

X_i is a feature (column) vector.
 x_i is a row vector (ith training example).

- **The equations in matrix form**

- $\begin{bmatrix} y_1 \\ \dots \\ y_N \end{bmatrix} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1k} \\ \dots & \dots & \dots & \dots \\ 1 & x_{N1} & \dots & x_{Nk} \end{bmatrix} \begin{bmatrix} w_0 \\ \dots \\ w_k \end{bmatrix} \Rightarrow y = Xw \text{ or } y = w^T X$
- $\underbrace{\quad}_Y \quad \underbrace{\quad}_X \quad \underbrace{\quad}_w$



The Goal of Regression Modeling

- **Housing price data set**

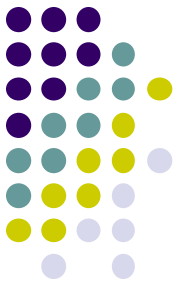
<i>Price (K\$) ($Y$)</i>	<i>Size (X_1)</i>	<i>Bedroom (X_2)</i>	<i>Bathroom (X_3)</i>	<i>Built year (X_4)</i>
375 (y_1)	1024 (x_{11})	3 (x_{12})	2 (x_{13})	1978 (x_{14})
425 (y_2)	1329 (x_{21})	3 (x_{22})	5 (x_{23})	1992 (x_{24})
...	...	...	...	...
465 (y_N)	1893 (x_{N1})	4 (x_{N2})	4 (x_{N3})	1980 (x_{N4})

- **Housing price data set in matrix form**

- $$\begin{bmatrix} 375 \\ 425 \\ \dots \\ 465 \end{bmatrix} = \begin{bmatrix} 1 & 1024 & 3 & 2 & 1978 \\ 1 & 1329 & 3 & 5 & 1992 \\ \dots & \dots & \dots & \dots & \dots \\ 1 & 1893 & 4 & 4 & 1980 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_4 \end{bmatrix}, \text{ X and y are from training data (known); w are unknown.}$$

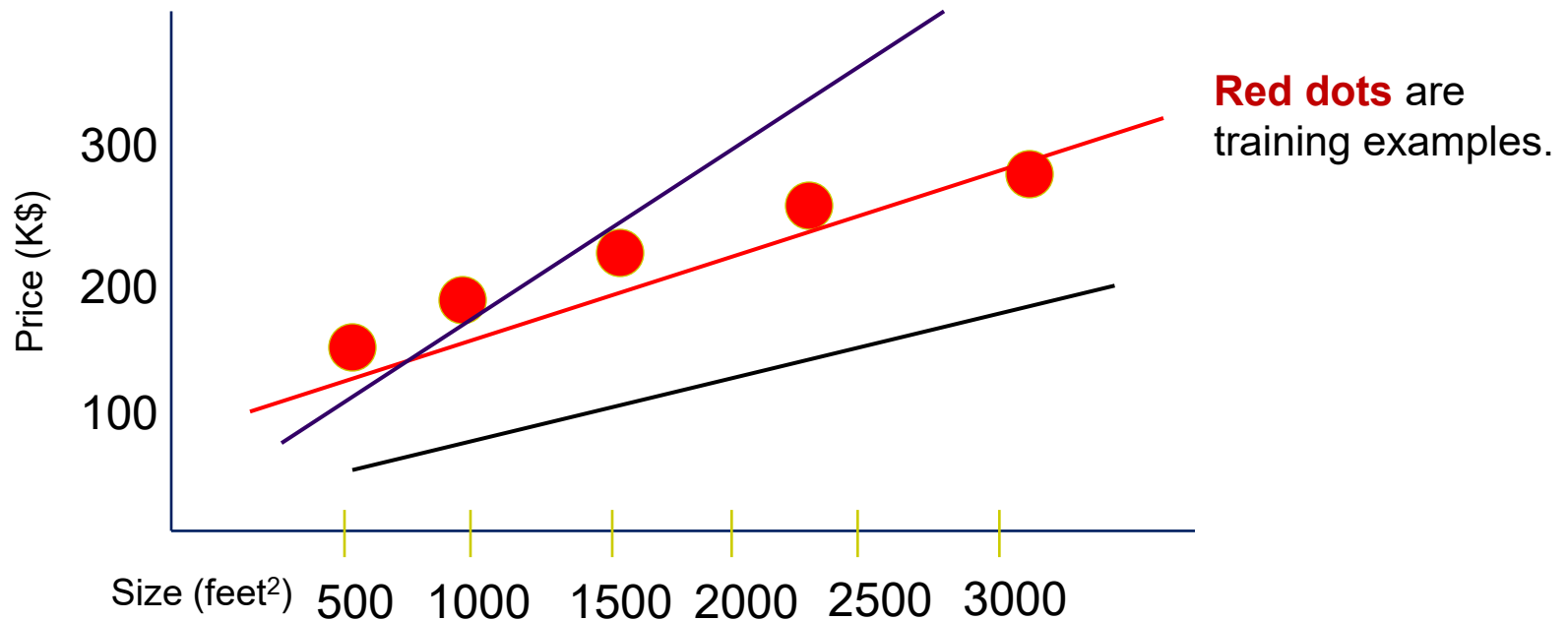
- **The goal is to estimate w of the function $f(X)$**

- So that $y = f(X) = Xw$
- With the estimated $\hat{w}$, for a new observation, x_{new} , we can **predict** y by $\hat{y} = \hat{w}^T x_{new}$.
- We want the predicted value $\hat{y} \approx$ actual value y .



How do we Find the Best Model?

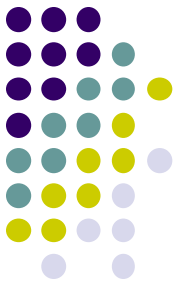
- We want the most accurate model with the least error in prediction.



- **One simple idea:**

- Choose the parameters $W = (w_0, w_1)$ so that $f(x)$ is closest to the actual values of y in the training examples (x, y) .

The Least Square Approach



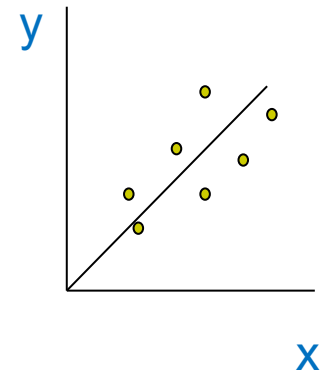
- The **basic idea** of the (ordinary) **least square approach**
 - Find **w** that minimizes the squared error:

$$\operatorname{argmin}_w \sum_i (y_i - \hat{w}^T x_i)^2$$

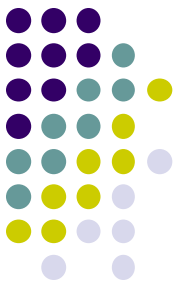
y_i is the actual value.

$\hat{w}$ is the estimated parameter.

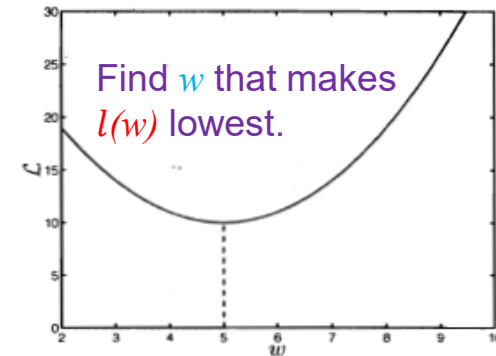
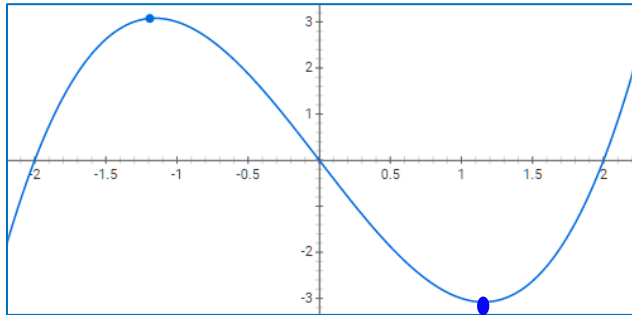
$\hat{w}^T x_i$ is the predicted value of y , $\hat{y}$ for a given data x_i .



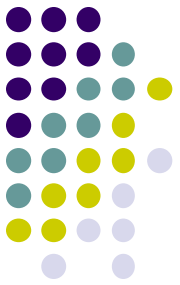
- Finding **w** with minimum **squared distances** between data in a training set and predicted line.
 - **Note:** The regression problem is **restated** as an **optimization problem** as an **error function**.



Derivatives to Test Turning Points



- The **first derivative** test:
 - Measures slope of the function at a point
 - A **local minimum** is where $f'(x)$ changes from **-** to **+**.
 - A **local maximum** is where $f'(x)$ changes from **+** to **-**.
 - **Solving** $f'(x) = 0$ returns the x-coordinates of **all turning points** (solutions).
- The **second derivative** test for **min** or **max** point:
 - Measures curvature of the function
 - If $f''(x) > 0$, the turning point at x is **convex**; $f(x)$ has a local **minimum** at x .
 - **Else if** $f''(x) < 0$, the turning point at x is **concave**; $f(x)$ has a local **maximum** at x .
 - **Else**: Saddle point (neither local minima nor maxima)



Linear Regression: $y = w^T X$

- Find parameters w that minimizes the loss function $l(w)$ for $\langle x, y \rangle$ value pairs of all the training examples (Gauss and Legendre, 1809).

$$\text{Minimize } l(w) = \frac{1}{N} \sum_i (y_i - f_w(x_i))^2$$

Squaring residuals is important!

N is the number of samples in a training data set.

- $f_w(x_i) = w^T x_i$
- Residual Sum of Squares (**RSS**) = $\sum_i (y_i - f_w(x_i))^2$
- Loss** or Mean Squared Error (**MSE**), $l(w) = \frac{1}{N} \text{RSS}$

$$\text{Argmin}_w l(w) = \frac{1}{N} \sum_i (y_i - w^T x_i)^2$$

- How can we find the lowest point of $l(w)$?

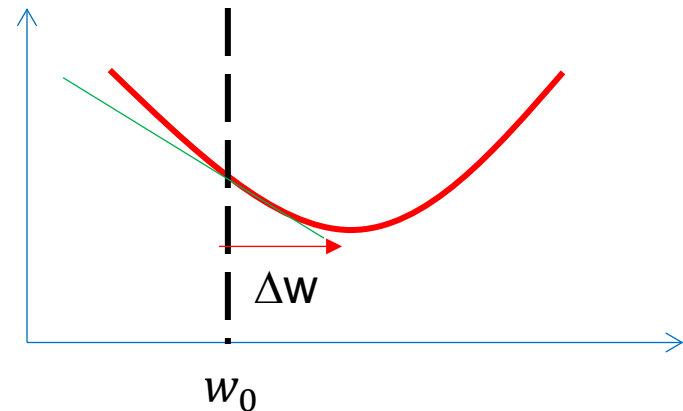
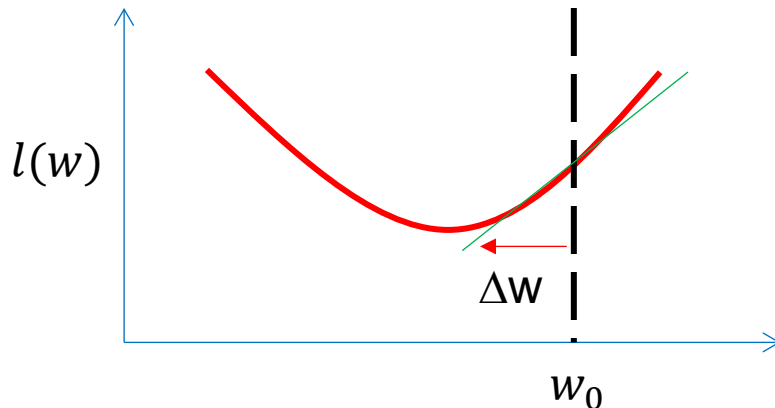
- Analytical solution: $\hat{w} = (X^T X)^{-1} X^T y$ or
- We can also find $\hat{w}$ iteratively.



Gradient Descent Method

- To **minimize** $l(w_0)$, we can use the following iterative method
 - Start** with an **initial** ($w_0 = \text{random}$)
 - Keep changing** w_0 **to reduce** $l(w_0)$: $w_{\text{next}} = w_0 \pm \Delta w$
 - Stop when** certain conditions are satisfied.

How to tell whether the **gradient** at a **point** w_0 is negative or positive?



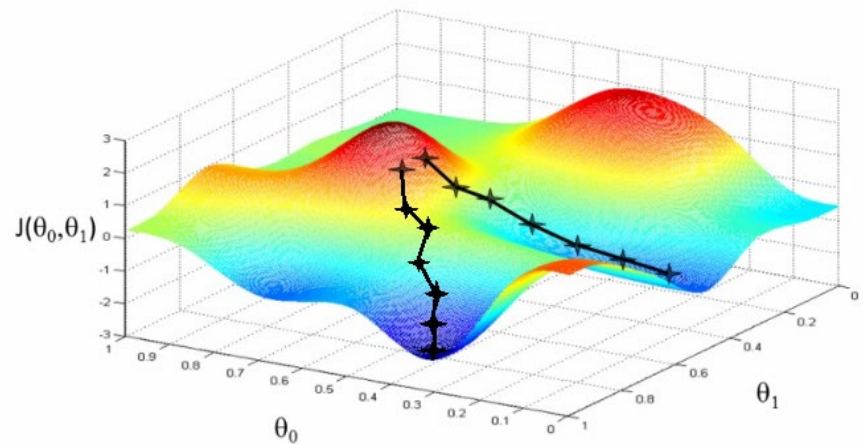
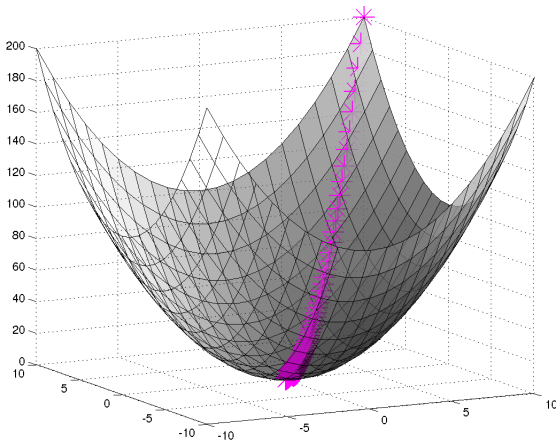
To **reduce** $L(w_0)$, **left case**: Δw should be positive, but **right case**: Δw should be negative.

How do we determine the sign of Δw (e.g., To change w_0 , subtract or add Δw)?

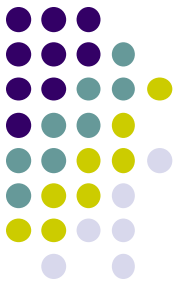
Linear Regression as a *Convex* Optimization Problem



Gradient descent (also called *steepest gradient descent*) is an **iterative optimization approach** to find the **minimum** or **maximum** of a function by taking **1st order derivative**.



Gradient descent will reach a **global optimum** if a function has only one optimum. Otherwise, it will reach a **local optimum**.



Gradient Descent Method

$$Y = w_0 X_0 + w_1 X_1 + w_2 X_2 + w_3 X_3 + \dots + w_k X_k \quad (X_0 = 1)$$

Initialize (w_j) with $k+1$ random values (k number of features)

Repeat until convergence {

for **each** w_j ($j=0, \dots, k$) in parallel

$$w_j = w_j - \eta \frac{1}{N} \sum_i (w^T x_i - y_i) x_{ij}$$

Update each w_j **simultaneously** (independently)

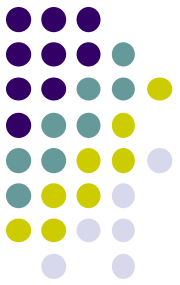
}

η is learning rate.

Each w_j is calculated by the **residual sum of all the N examples** in a training data set.

For example, we can compute w_0, w_1, w_2 :

- $w_0 = w_0 - \alpha \frac{1}{N} \sum_i (w^T x_i - y_i) x_{i0}$ ($x_{i0} = 1$)
- $w_1 = w_1 - \alpha \frac{1}{N} \sum_i (w^T x_i - y_i) x_{i1}$
- $w_2 = w_2 - \alpha \frac{1}{N} \sum_i (w^T x_i - y_i) x_{i2}$



Predicting a Value for New Data

- Given a **new sample** x_{new} , the **predicted value**, $\hat{y}_{new}$ from the **model** $\hat{w}^T x_{new}$ obtained from training is computed by:

$$\hat{y}_{new} = \hat{w}^T x_{new}$$

- A value of y_i is computed by **dot product** of w^T and x_i , that is, $y_i = w^T x_i$

$$y_i = w_0 + w_1 x_{i1} + w_2 x_{i2} + \dots + w_k x_{ik}$$

When to Update Gradients in Training



- **Pretraining**

- Train with the **initial weights learned from some examples** and fine-tune the model.

- **Sequential training**

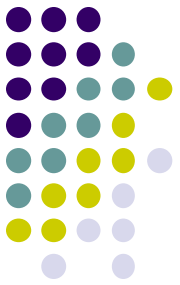
- **The weight update for each input example.** Slow convergence but may avoid local minima, potentially reaching better solutions.

- **Batch training**

- **All the training examples are fed to the network. The weight update for each epoch for the average error.** Faster convergence to a local minimum since the weights are moved in the direction that most of the inputs want them to move.

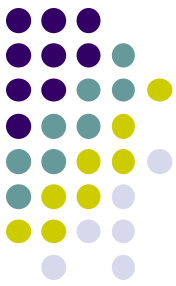
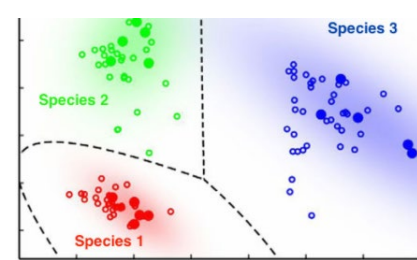
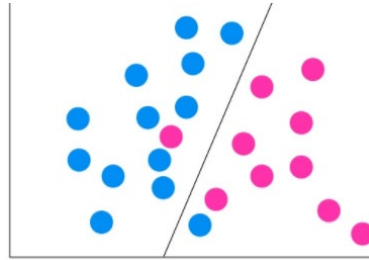
- **Stochastic Gradient Descent and Minibatches**

- Stochastic Gradient Descent (SGD)
 - The weights are updated for each one randomly selected input. Repeat the process until convergence. Often used when the training set is very large.
- **Minibatches Gradient Descent** (MGD, combination of batch and SGD)
 - The training set are split into fixed-size or random batches, then update the weights for each batch.
 - The training set are then randomly shuffled into new batches and the process is repeated.
 - If the batches are small, then the global minimum may be found although at the cost of heading in the wrong direction.



Supervised Learning for Classification

Classification



- **Types of classification**

- Binary class and multiclass (by the number of classes)
- Linear and nonlinear (by the type of boundary function)

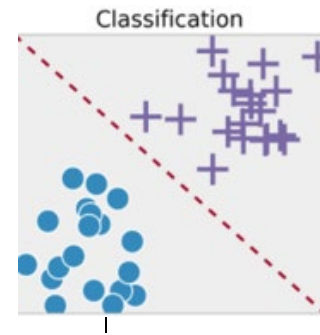
- **Numerous applications**

- Pattern recognition, object detection, text classification, speech recognition, language modeling, etc.

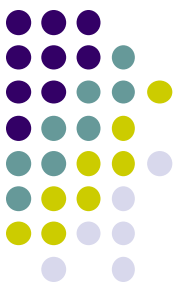
- **Various classification algorithms**

- Bayes, Naïve Bayes, Logistic Regression, Artificial Neural Networks, Support Vector Machine, K-Nearest Neighbors, Decision tree induction, etc.

Supervised Learning: Classification



- Given a training data X with K features and target y $\{X_1, X_2, \dots, X_k, y\}$
 - Learn a **classifier** $f: x \rightarrow y$ such that $f(x)$ is able to determine a discrete or categorical value y called “**class**” labels ($y_i \in \{C_1, C_2, \dots, C_k\}$) for an example x in X .
- Two major ideas for classification**
 - Find the **model parameter** θ that **maximizes** the likelihood:
 - $\theta^* = \operatorname{argmax}_{\theta} \prod P(y_i|x_i; \theta)$ or $\theta^* = \operatorname{argmax}_{\theta} - \sum \log P(y_i|x_i; \theta)$
 - After training, predict the label for a new data x by choosing the class with the highest posterior probability (**MLE**): $\hat{y} = \operatorname{argmax}_{c \in \{C_1, C_2, \dots, C_k\}} P(y = c|x)$
 - Baye's rule** can be used for estimating the likelihood.
 - Find the **decision boundary function** that **separates** the different groups of data.
 - A decision boundary is the **hyperplane** that separates class regions.
 - Logistic Regression, Neural Networks, etc. can be used to find the decision boundary.
 - These two ideas are mathematically connected.
 - A decision boundary is where classes have equal probability.



Maximum Likelihood Classification

- **Bayesian rule**

$$P(Y = C_j|X) = \frac{P(X|Y = C_j)P(Y = C_j)}{P(X)} = \frac{P(X|Y = C_j)P(Y = C_j)}{\sum_{m=1}^M P(X|Y = C_m)P(Y = C_m)}$$

- Two popular **MLE options** for **prior** class distribution if **prior** is **not** known
 - Uniform prior $P(Y = C_j) = 1/M$ where M is the number of classes
 - Class size prior $P(Y = C_j) = C_{jN}/N$ where N is the # of examples in the training data and C_{jN} is the # of examples belonging to class C_j in the data set.
- Marginal likelihood can be ignored.
- Posterior probability can be estimated by **Likelihood*Prior**.
- Therefore, classification is defined as a probability distribution over class C.

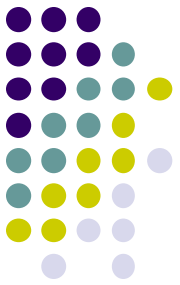
- **Training**

- For each class C_j , estimate $P(X|C_j)$ and $P(C_j)$:
 - If x is categorical, just count values.
 - If x is continuous, either discretize or estimate predetermined distribution such as Gaussian.

- **Bayesian classification**

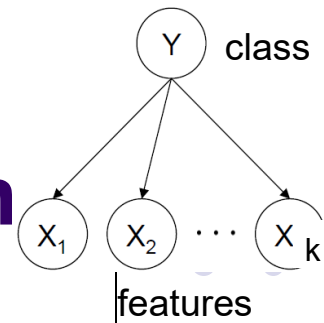
- For a new example x_{new} , choose the **class** C_j that **maximizes** $P(x_{new}|C_j)P(C_j)$. 67

Problems of Bayesian Classification

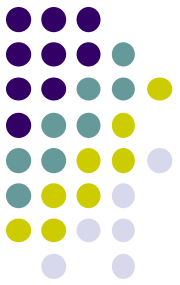


- Require probability distribution that is unknown in many other cases.
- Probability calculation can be expensive as the size of data increases.
 - **Joint distribution** for model parameters with **K features**, for **posterior**, $\operatorname{argmax}_y P(y|X_1, X_2, \dots, X_k)$, the **likelihood**, $P(X_1, X_2, \dots, X_k|y)$
$$\begin{aligned} P(X_1, X_2, \dots, X_k|y) &= P(X_1, X_2, \dots, X_k, y) = P(X_1|X_2, \dots, X_k, y)P(X_2, \dots, X_k, y) \\ &= P(X_1|X_2, \dots, X_k, y)P(X_2|X_3, \dots, X_k, y)P(X_3, \dots, X_k, y) = \dots \\ &= P(X_1|X_2, \dots, X_k, y)P(X_2|X_3, \dots, X_k, y)P(X_3, \dots, X_k, y) \dots P(X_{k-1}|X_k, y)P(X_k|Y)P(y) \end{aligned}$$

The Naïve Bayes (NB) Classification

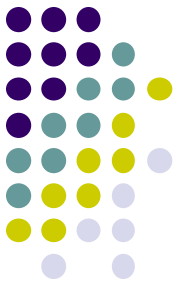


- Let's assume **features are independent given a class**.
 - $P(X_1, X_2 | Y) = P(X_1 | X_2, Y)P(X_2 | Y) = P(X_1 | Y)P(X_2 | Y)$
 - More generally $P(X_1, X_2, \dots, X_k | Y) = \prod_i P(X_i | Y)$
 - $P(C_j | X_1, X_2, \dots, X_k) \propto P(C_j, X_1, X_2, \dots, X_k) = \propto P(C_j)P(X_1 | C_j)P(X_2 | C_j)$
by *chain rule* ... $\propto P(C_j) \prod_{i=1}^K P(X_i | C_j)$
- Naïve Bayes classification**
 - Given prior $P(C_j)$, K conditionally independent features $X_1, X_2, \dots, X_k$, the posterior probability by MLE or MAP estimation:
 $\hat{c} = \underset{C_j}{\operatorname{argmax}} P(C_j) \prod_{i=1}^K P(X_i | C_j)$ (Ignoring the marginal likelihood)
 - Are the Naïve Bayes assumptions realistic?**
 - Yes, in many cases, e.g., for features of **Apple(shape, color, size)**, each of these features contributes independently to the probability that the fruit is an apple.



How to Calculate the Likelihood

- Calculating the likelihood: $\operatorname{argmax}_{C_j} P(C_j) \prod_{i=1}^K P(X_i|C_j)$
 - If x is **categorical**, just count, $P(X_i|C_j) = \frac{\text{count of } X_i \text{ in } C_j}{\text{total count of instances in } C_j}$.
 - If x is **continuous**, either **discretize** or **estimate** predetermined distribution with PMF or PDF of the distribution.
 - Gaussian Naïve Bayes: $P(x|C_j) = \frac{1}{\sigma_j \sqrt{2\pi}} \exp\left(-\frac{(x-\mu_j)^2}{2\sigma_j^2}\right)$
 - Calculate μ_k and σ^2 in X^c (X^c is X associated with class C_k).
 - Bernoulli Naïve Bayes: $P(x|C_j) = \prod_{i=1}^K p_{ij}^{x_i} (1 - p_{ij})^{1-x_i}$
 - Discretize the values to obtain a new set of Bernoulli-distributed features.



The Naïve Bayes Classification

- **For training**

- Calculate and store likelihood for each attribute and each class $P(x_i|C)$
- Store prior probabilities of all the classes $P(C)$

- **For prediction**

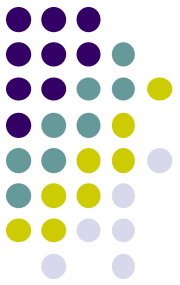
- Retrieve the precomputed likelihoods and priors for each class
- Use these stored values to calculate the posterior probability for the new example by multiplying the relevant likelihoods $P(C|X)$ and prior probabilities
- Predict the class by comparing posteriors, $MLE_c P(C|X)$

Example: Naïve Bayes Classification



- Will a customer with the following profile
“age<30, income=medium, student=Yes” **buy a computer?**
- Based on the naïve Bayes
 - $P(\text{Bought}=\text{yes})=9/14=0.643$, $P(\text{Bought}=\text{no})=5/14=0.357$
 - $P(\text{Age}<30 | \text{yes})=2/9=0.222$, $P(\text{Age}<30 | \text{no})=3/5=0.6$
 - $P(\text{Income}=\text{medium} | \text{yes})=4/9=0.444$,
 $P(\text{Income}=\text{medium} | \text{no})=2/5=0.4$
 - $P(\text{Student}=\text{yes} | \text{yes})=6/9=0.667$,
 - $P(\text{Student}=\text{yes} | \text{no})=1/5=0.2$
- Calculated probabilities:
 - $P(X | \text{yes}) = 0.222 * 0.444 * 0.667 = 0.066$.
 - $P(X | \text{no}) = 0.6 * 0.4 * 0.2 = 0.048$.
 - $P(X | \text{yes})P(\text{yes}) = 0.066 * 0.643 = 0.042$.
 - $P(X | \text{no})P(\text{no}) = 0.048 * 0.357 = 0.017$
- Answer:
 - By the estimation, the naïve Bayes classifier predicts the customer will “buy” a computer.

Age	Income	Student	<u>Bought</u>
20	High	Yes	Yes
25	Low	No	No
50	High	No	Yes
70	Medium	Yes	Yes
15	Medium	Yes	Yes
30	High	No	No
10	Low	Yes	No
40	Medium	Yes	Yes
35	Medium	Yes	Yes
41	Medium	No	No
45	High	Yes	Yes
22	Medium	No	No
55	High	No	Yes
50	High	No	Yes



Handling Missing Data

- **No values for some attribute**

- E.g., applicant's credit history unknown, a medical test not performed on a patient
- How do you compute $P(X_1 = x_1 \dots X_j = ? \dots X_k = x_k | y)$?

- **Handling missing data**

- Ignore the examples with missing values:

$$P(x_1 \dots X_j \dots x_k | y) = \prod_{i \neq j}^k P(x_i | y)$$

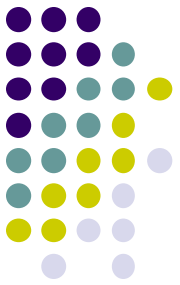
- Computing the likelihood for X_j with a missing value

$$P(x_1 \dots X_j \dots x_k | y) = P(x_1 | y) \dots P(X_j | y) \dots P(x_k | y) = P(x_1 | y) \dots 1 \dots P(x_k | y)$$

- **Example:** Coin tossing

- Event = $\{X_1=H, X_2=?, X_3=T\}$, ignore X_2 by setting 1.
- Event = $\{X_1=H, X_2=H, X_3=T\} + \{X_1=H, X_2=T, X_3=T\}$

- **Being able to handle missing values is important!**



The Naïve Bayes with Smoothing

- **Problem of multiplication by zero**

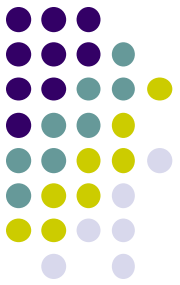
- **Multiplying any term with zero** will wipe out all the information in other probabilities when they are multiplied.
 - This is quite possible when a particular value doesn't appear in a class.

- **Some solutions**

- **Laplace soothing** (or additive soothing, regularizing naïve Bayes)
 - Incorporate a small-sample correction in all probability estimates such that no probability is set to be zero:

$$\hat{\theta}_i = \frac{x_i + \alpha}{N + \alpha K} \text{ where } K \text{ is \# of features}$$

- **Placing a prior density** that encodes the belief that all probabilities are greater than 0.
 - **Dirichlet density** is a suitable prior density for a vector of probabilities. It is the multivariate generalization of the Beta distribution and the conjugate prior of the categorical or multinomial distribution.



Strengths and Weakness of NB

● Strengths

- Highly scalable
- Can handle both real and discrete data including streaming data
- Not sensitive to irrelevant features and can handle missing values
 - Text document classification, SPAM filtering, this assumption works reasonably well.

● Weaknesses

- When the independence assumption does not hold, it may not work.
 - e.g., NB cannot model any within-class dependencies.

- Naïve Bayes is a linear classifier:

- Theorem: Assume that $y_i \in \{0, 1\}$. Then the Naïve Bayes classifier is defined by:

$$h(x) = \operatorname{argmax}_y P(y) \prod_j P(x_j|y) = \text{sign}(w^T x)$$

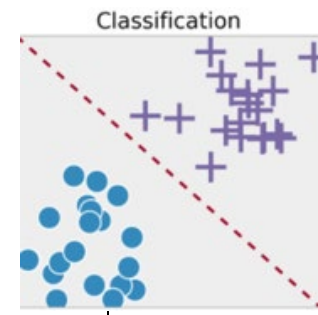
- We can show that: $w^T x > 0 \iff h(x) = +1$

- For continuous features (Gaussian Naïve Bayes), we can use:

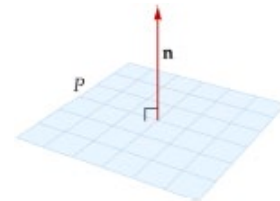
$$P(y|x) = \frac{1}{1 + e^{-y(w^T x + w_0)}} \text{ (This is logistic regression.)}$$

- The Naïve Bayes and logistic regression produce asymptotically the same model if the Naïve Bayes assumption holds.

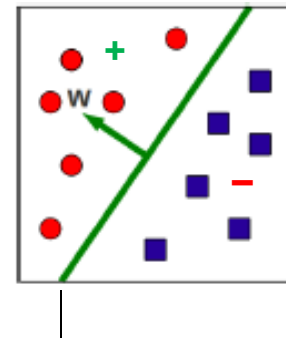
Decision Boundary for Classification



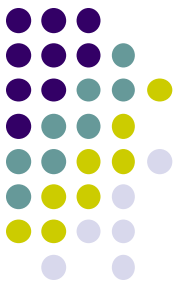
- Given a training data X with K features and target y $\{X_1, X_2, \dots, X_k, y\}$
 - Learn a **classifier** $f: x \rightarrow y$ such that $f(x)$ is able to determine a discrete or categorical value y called “**class**” labels ($y_i \in \{C_1, C_2, \dots, C_k\}$) for an example x in X .
- Find the **decision boundary** that **separates** the different groups of data as clearly as possible.
 - A decision boundary is the hyperplane that separates class regions.
 - Logistic Regression, Neural Networks, etc. can be used to find the decision boundary.
- How to find a decision boundary**
 - Find a *hyperplane* that separate the class regions.
 - A **plane** is a flat surface that extends infinitely without thickness.
 - A **hyperplane** is a flat subspace whose dimension is one less than that of its ambient space and divides the space into **two half-spaces** (or **two sides**), acting like a “**boundary**”.
 - How can we determine the class label** ($\pm$, 1/0, or $y_i \in \{C_1, C_2, \dots, C_k\}$)?
 - Which side is positive or zero?**



Decision Boundary for Classification



- **Use a hyperplane as decision boundary**
 - A hyper plane can separate **n-dimensional space** into **two half-spaces**.
 - A **hyperplane** defined by a **linear discriminant function** $f(x) = w^T x + w_0$
 - w is the **normal vector** to the **hyperplane** defined by $w^T x + w_0$
 - w_0 is the bias (or threshold) that represents the **position** of the **hyperplane** from the **origin**.
- **The classification problem** from geometric perspective
 - Learns w that determines the **direction** of the decision boundary from the training data.
 - For **binary classification**, the **direction** of the vector w can be the sign either **+** (outward of the vector) or **-** (opposite side).
 - If $w^T x + w_0 \geq 0$ ($w^T x \geq w_0$ or $y \geq 0$), **+** class
 - If $w^T x + w_0 < 0$ ($w^T x < w_0$ or $y < 0$), **-** class
 - If $w^T x + w_0 = 0$, **data points** are **on** decision boundary (a **hyperplane**).



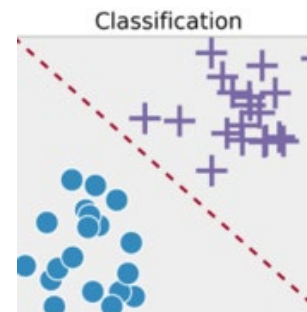
Solving the Classification Problem

Training dataset

Label: Binary
classification (1/0)

Predictors				Target
Outlook	Temp.	Humidity	Windy	Play Golf
Rainy	Hot	High	False	No
Rainy	Hot	High	True	No
Overcast	Hot	High	False	Yes
Sunny	Mild	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Overcast	Cool	Normal	True	Yes
Rainy	Mild	High	False	No
Rainy	Cool	Normal	False	Yes
Sunny	Mild	Normal	False	Yes
Rainy	Mild	Normal	True	Yes
Overcast	Mild	High	True	Yes
Overcast	Hot	Normal	False	Yes
Sunny	Mild	High	True	No

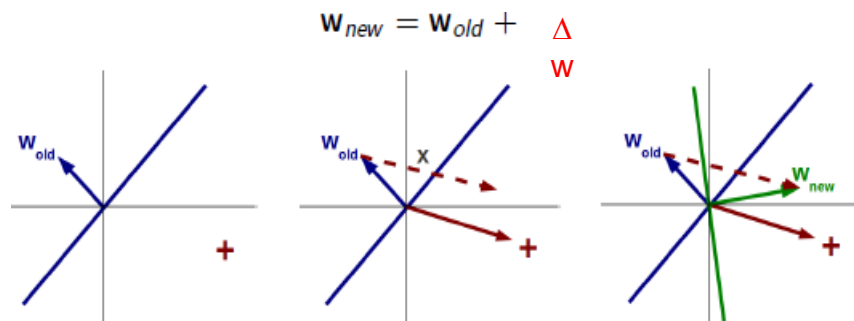
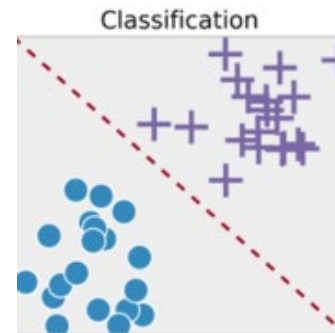
From the **linear model** $y = w^T X$ (the **linear combination** of the input features), we want y value to be **0/1** for **binary classification**.



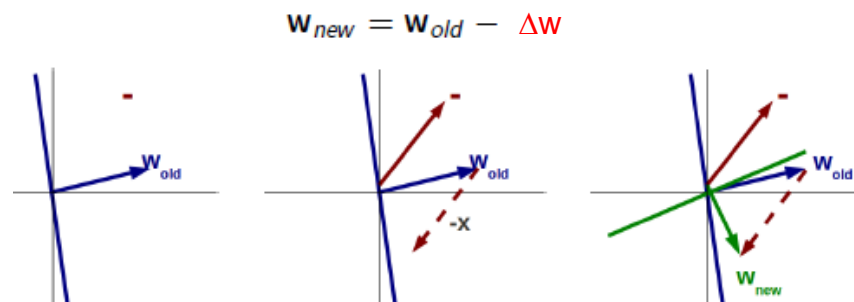
- Find w that correctly classifies the data based on the label values in the training dataset.
 - Find w that the positive direction is correctly aligned with the positive classes in the training dataset.

Geometric View of Learning

Correct **W** for Classification



A misclassified + example
Incorrectly thought $w^T x < 0$



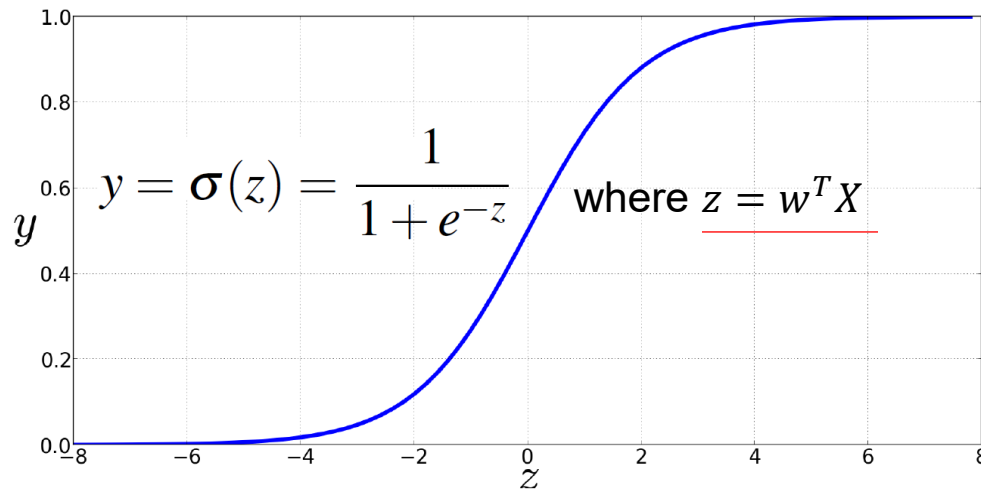
A misclassified - example
Incorrectly thought $w^T x > 0$

The **dot product** $\hat{w}^T x$ (defines **decision boundary** or **hyperplane**) can be considered the **projection** of the **input vector** x **onto** the vector $\hat{w}$. The **vector** $\hat{w}$ points in a direction of class 1 and **perpendicular** to the **decision boundary** $\hat{w}^T x = 0$.

The **sign** of $\hat{w}^T x$ determines on **which side** of the hyperplane an input feature x lies (+ or - side, classifying it as 1/0).

We need to **find correct w** by updating w_{old} using an **update rule** that corrects the error in the correct **gradient direction** by changing the decision boundary line and the orthogonal vector w . **This is what will happen in training.**

Classification with Logistic Function



Useful logistic function

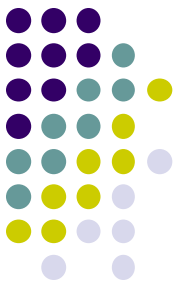
- The value of $\sigma(z)$ is normalized **[0,1]** – can be used for a probability value.
- A continuous function is differentiable at each point.
- The value of derivative is greatest where the function is changing most rapidly.
- If $\hat{w}^T x$ is positive and large, the logistic function will output a value close to 1, indicating high probability of the positive case.

$$P(y = 1) = \sigma(w^T X) = \frac{1}{1 + e^{-w^T X}}$$

$$P(y = 0) = 1 - \sigma(w^T X) = 1 - \frac{1}{1 + e^{-w^T X}} = \frac{e^{-w^T X}}{1 + e^{-w^T X}}$$

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) \geq 0.5 \text{ (when } w^T X \geq 0) \\ 0 & \text{Otherwise (when } w^T X < 0) \end{cases}$$

We need to find w for classification, but how do we find it?



Learning w by $\hat{w}_{MLE}$ in Training

- Deriving a gradient and update rule

- From the log loss function $H(y, \hat{y}) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y}))$

$$\text{where } \hat{y} = \frac{1}{1 + e^{-\hat{w}^T x}}$$

- The derived gradient (The detailed derivation steps are skipped.)

$$\frac{\delta H(y, \hat{y})}{\delta w_j} = [\hat{y} - y]x_j = (\hat{y} - y)x_j \text{ where } x_j \text{ is the value of } j\text{th feature of } x.$$

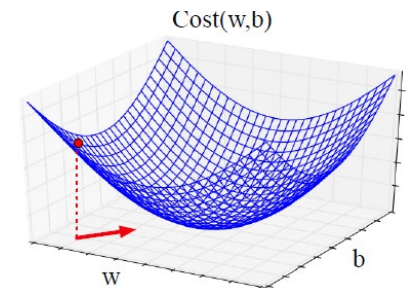
$$w_j = w_j - \alpha \frac{\delta H(y, \hat{y})}{\delta w_j}, w_j = w_j - \alpha (\hat{y} - y)x_j \text{ (We have seen this before.)}$$

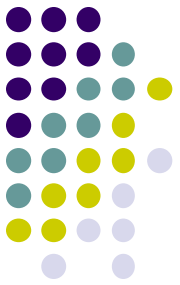
$$\frac{\delta H(y, \hat{y})}{\delta w_j} = \frac{1}{N} \sum_{i=1}^N [\hat{y}_i - y_i]x_{ij} = \frac{1}{N} (\hat{y} - y)^T X \text{ (using the entire data set)}$$

- Update rule: $w_j = w_j - \alpha \frac{1}{N} [\sum_{i=1}^N (\hat{y}_i - y_i)x_{ij}]$

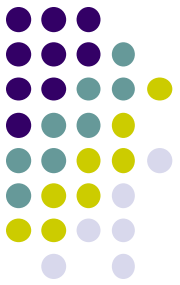
- Training for $\hat{w}_{MLE} = \operatorname{argmin}_w H(y, \hat{y})$

- Start with an initial w (randomly generated)
- Compute predicted $\hat{y}$ using the logistic function $\sigma(z)$ where $z = w^T X$
- Adjust w using the update rule until converged



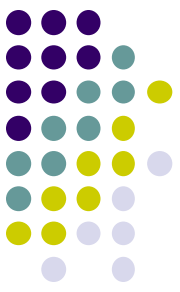


Artificial Neural Networks



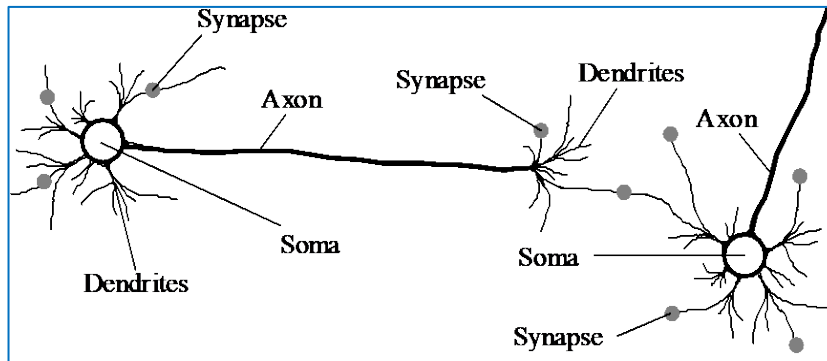
Bio-inspired Computing Topics

- **Sensory organs:** Perception, Vision, Sensor Networks
- **Immune system:** Artificial Immune Systems
- **Life:** Cellular Automata, Artificial Life
- **Brain** and **nerve system:** Artificial Neural Networks
- **Cognition:** Cognitive Systems
- **Agent:** Humanoid robots, bio robots, autonomous cars, drones, etc.
- **Group behavior:** Swarm Intelligence in Ants, Bees, Birds, etc.
- **Evolution:** Genetic Algorithm, Genetic Programming
 - Living species have evolved to be natural for millions of years.

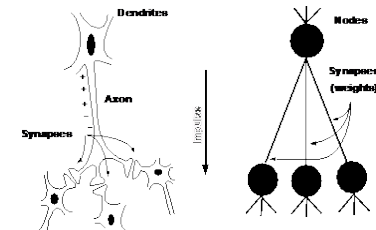


Biological Neuron vs Artificial Neuron

- From biological neuron (specialized cell) to artificial neuron



Extremely simplified model



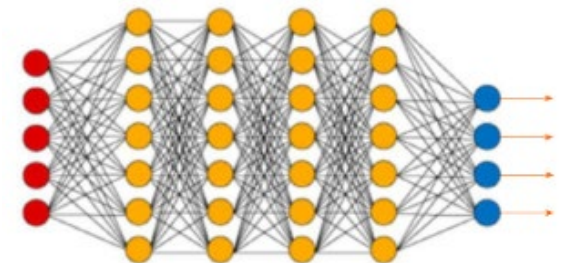
- Human brain**

- Number of neurons : $\sim 8 \times 10^{10}$ (>85 billions)
- Connections per neuron: $\sim 10^4$ -5
- Scene recognition time: ~ 0.1 second
- Cells that fire together, wire together (Hebbian learning)

- Artificial Neural Network (ANN)**

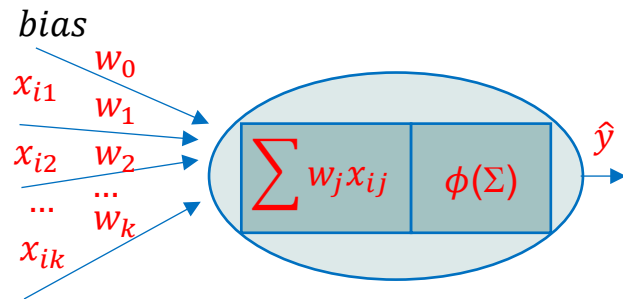
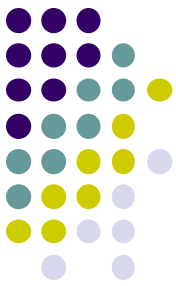
- An **artificial neuron** is a basic building block of ANN.
- Many neurons like threshold switching units
- Many weighted interconnections among units
- Highly parallel and distributed processes

Complex model



Artificial Neuron Model

(McCulloch-Pitts' Neuron Model, 1943)



Computing a logic function **AND**

Bias=1, $w=[-2,1,1]$, $\phi(\Sigma)$:

$\hat{y} = 1$ if $\Sigma \geq 0$ (threshold) else 0

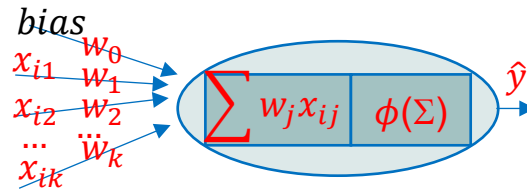
x_1	x_2	$x_1 \text{ AND } x_2$	Σ	$\hat{y} = \phi(\Sigma)$
1	1	1	0	1
1	0	0	-1	0
0	1	0	-1	0
0	0	0	-2	0

- **Each neuron** consists of
 - **Input signals** as a vector x from environment (data) or other neurons
 - **Bias** is usually necessary when all inputs are zero. That's exactly the purpose of intercept in a function.
 - **Weights** as a vector w representing the connection strengths
 - **Activation function**, $\phi(\Sigma)$ takes the weighted sum $\Sigma = \sum w_i x_i$ and determines the neuron's output $\hat{y}$

If $w=[-1,1,1]$, the neuron can compute **OR**.

Perceptron: Learning the Weights

(Rosenblatt, 1958)



The Perceptron Algorithm:

Given a set of N training examples $\{(x_1, y_1), \dots, (x_N, y_N)\}$

Initialize $w = [w_0, w_1, \dots, w_k]$ and learning rate η

Repeat until convergence

for each input vector x_i ($i = 1, \dots, N$)

$$w^{t+1} = w^t - \eta(\hat{y}_i - y_i)x_i$$

return w

● Setting up for training

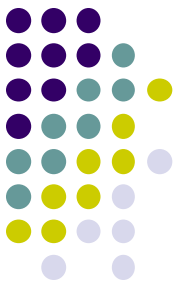
- **Gradient** is $(\hat{y} - y)x$ from $l(\hat{y}_i, y_i) = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$
- **bias**=1, **Initial** $w = [-0.6, 0.75, 0.5]$, $\alpha = 0.2$
- **Activation function** $\phi(\Sigma)$: 1 if $\Sigma \geq -0.6$ (w_0) else -1

● Training result

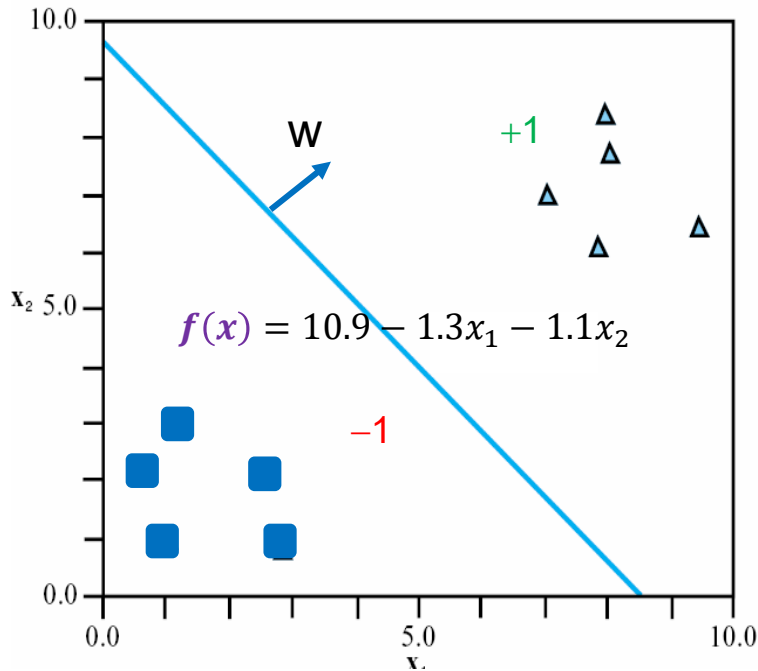
- After 500 iterations, it finds $w = [10.9, -1.3, -1.1]$

Example data

X_1	X_2	y
1.0	1.0	1
9.4	6.4	-1
2.5	2.1	1
8.0	7.7	-1
0.5	2.2	1
7.9	8.4	-1
7.0	7.0	-1
2.8	0.8	1
1.2	3.0	1
7.8	6.1	-1

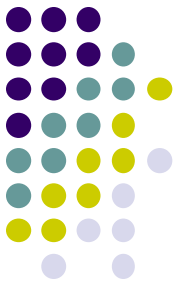


The Capability of the Perceptron

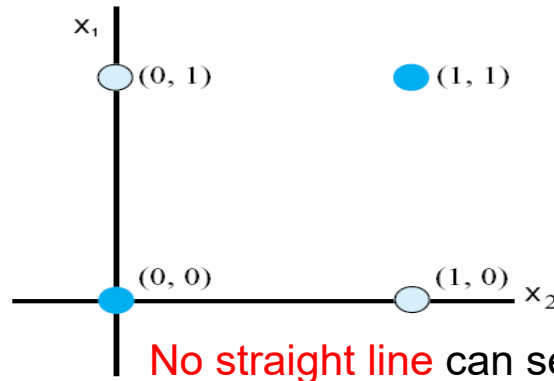


- The **linear discriminant function** $f(x)$ defines the **decision boundary** that linearly separates the data set into **+1** or **-1**.
 - $f(x)$ is a **hyperplane**.
 - $w = [-1.3, -1.1]$ is the **normal vector**.
- Will the Perceptron find the **best** decision boundary?
 - **NO**, it will find the first one and stop as soon as it gets all the training examples correct (**local minimum**).
- Convergence of Perceptron
 - (Block & ovikoff, 1962) **proved** that the **Perceptron** will **converge** **IF** the training data is **linearly separable**, and the learning rate is sufficiently small.
 - Convergence rate does not depend on the # of training examples or dimensions, but **the margin!**
- How about linearly NOT separable data?

Linearly NOT Separable Case: XOR

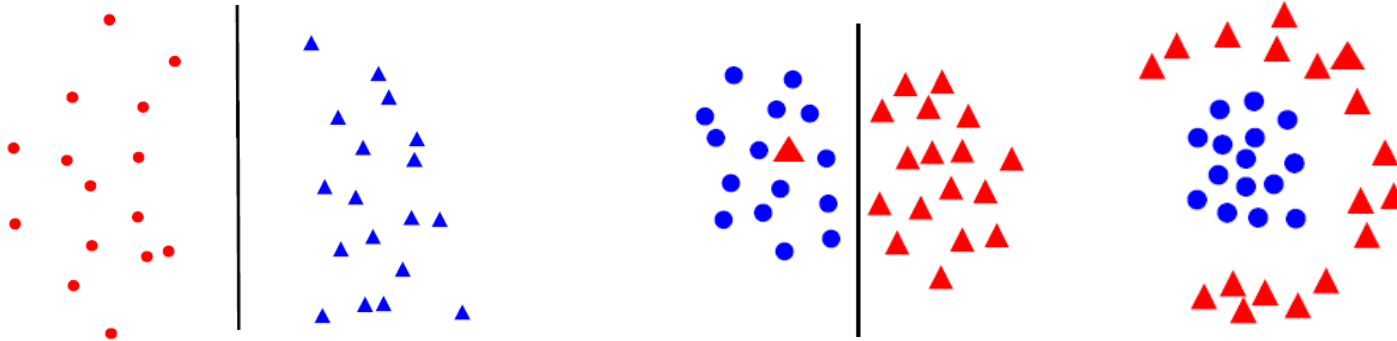
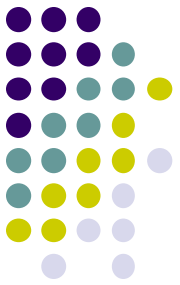


x_1	x_2	$x_1 \text{ XOR } x_2$
1	1	0
1	0	1
0	1	1
0	0	0



- (Minsky and Papert, 1969) proved that a **perceptron cannot** solve the **XOR problem**. From the above data set, following equations must be solved:
 - $w_1 * 1 + w_2 * 1 < t$ where t is a threshold value.
 - $w_1 * 1 + 0 \geq t$
 - $0 + w_2 * 1 \geq t$
 - $0 + 0 < t$ ($t > 0$)
- These equations has no solution.
- **Impact:**
 - The book somehow discouraged researchers to further work on the algorithm until 1980s. ⁸⁸

Linearly Separable and Not Separable



- **Linearly separable**

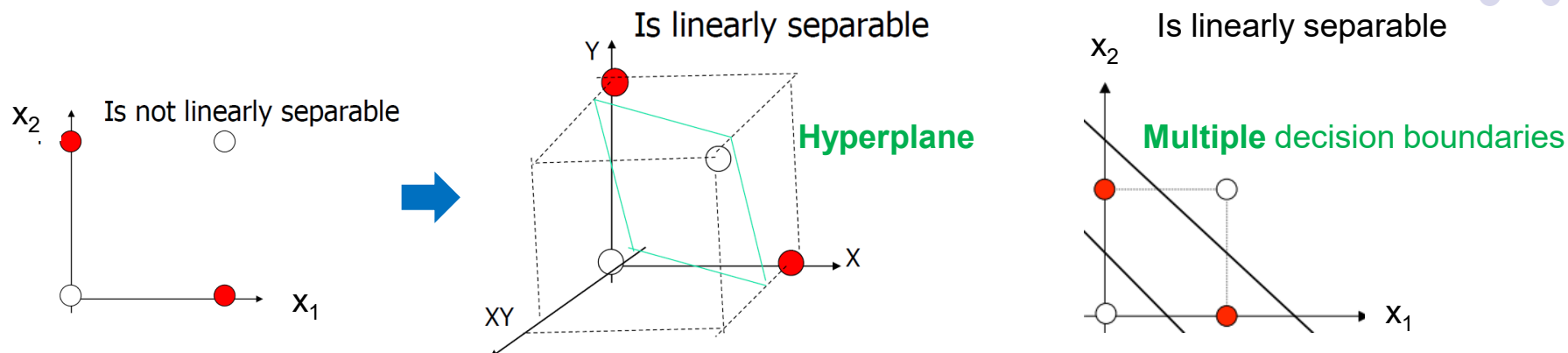
- A linear function exists to separate the data.

- **Linearly not separable**

- No linear function exists to separate the data.

- Is it possible to use a **linear decision boundary** to separate non-linearly separable data?

Solving Nonlinearly Separable Problems



- **Transforming data**

- Transform the data to a high dimensional using a **Kernel function**.
 - Support Vector Machine (SVM) is based on hyperplane and Kernel.

- **Adding more neurons and/or multilayer perceptron**

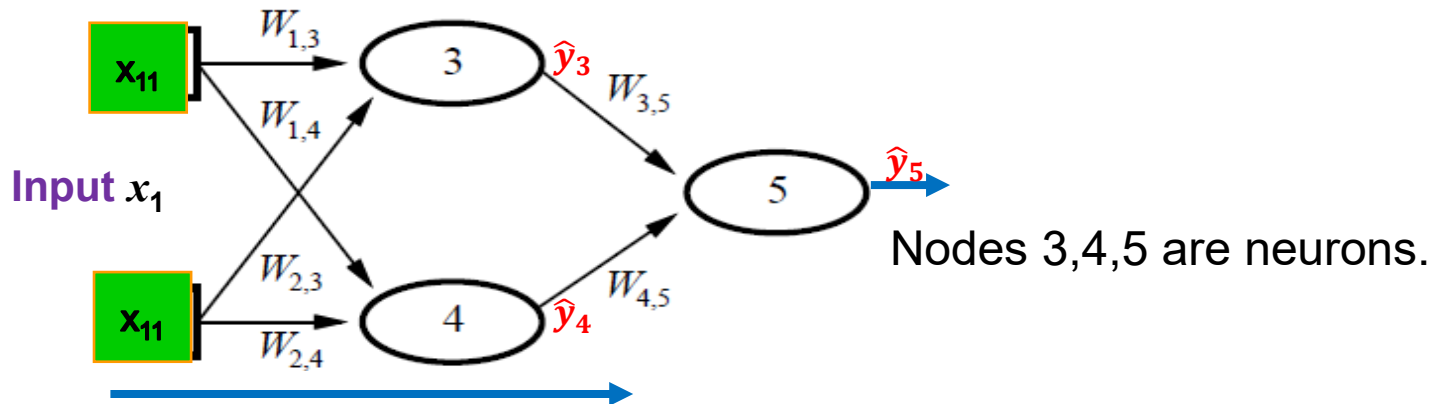
- Complex neural network: multiple neurons or multilayer perceptron
 - With multiple neurons multiple decision boundaries: **Each neuron has its own decision boundary.**

- Using **complex** (non-linear) **activation functions**

- Sigmoidal function $\sigma(\sum w_i x_i) = \frac{1}{(1+e^{-\sum w_i x_i})} = \frac{1}{(1+e^{-w^T x})}$

The Multilayer Perceptron (MLP)

(The idea first introduced by **Worbois**, 1974. Later rediscovered by **Rumelhart** et al.1986)

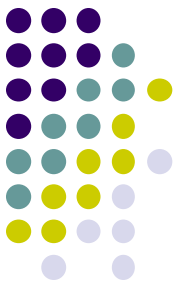


- **Multilayer *feedforward* perceptron**

- More neurons naturally creates multiple layers and creates a network of neurons (neural network).
- Inputs are fed into a network propagating to the output node (feedforward).

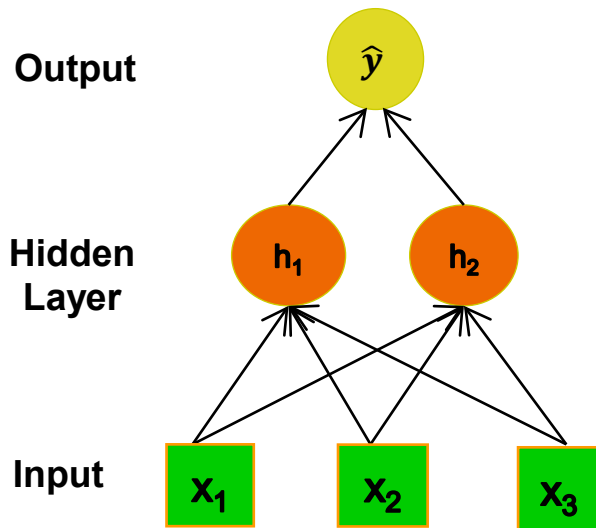
- **The network** is now a parameterized family of **nonlinear functions**.

- $\hat{y}_3 = g(w_{1,3}x_{11} + w_{2,3}x_{12})$, $\hat{y}_4 = g(w_{1,4}x_{11} + w_{2,4}x_{12})$, $\hat{y}_5 = g(w_{3,5}\hat{y}_3 + w_{4,5}\hat{y}_4)$
- $\hat{y}_5 = g(w_{3,5} \cdot g(w_{1,3}x_{11} + w_{2,3}x_{12}) + w_{4,5} \cdot g(w_{1,4}x_{11} + w_{2,4}x_{12}))$



A Forward Pass in a Neural Network

Neural architecture



- For multiple input examples, feedforward can be computed efficiently using **matrix operations**, $y = w^T x$

Using NumPy

```
# MLP feedforward with three-layer neural network
import numpy as np

# generating one training example
x = np.random.randn(3)
y = 0.7 # target value
# Biases
b1 = b2 = 1.0
x = np.append(b1, x)

# initial weight vectors W=[w_0, w_1, w_2, w_3]
# w_0 is the weight for the bias
W1 = np.array([0.5, 0.5, 0.5, 0.5])
W2 = np.array([0.5, 0.5, 0.5, 0.5])
W3 = np.array([0.5, 0.5, 0.5])

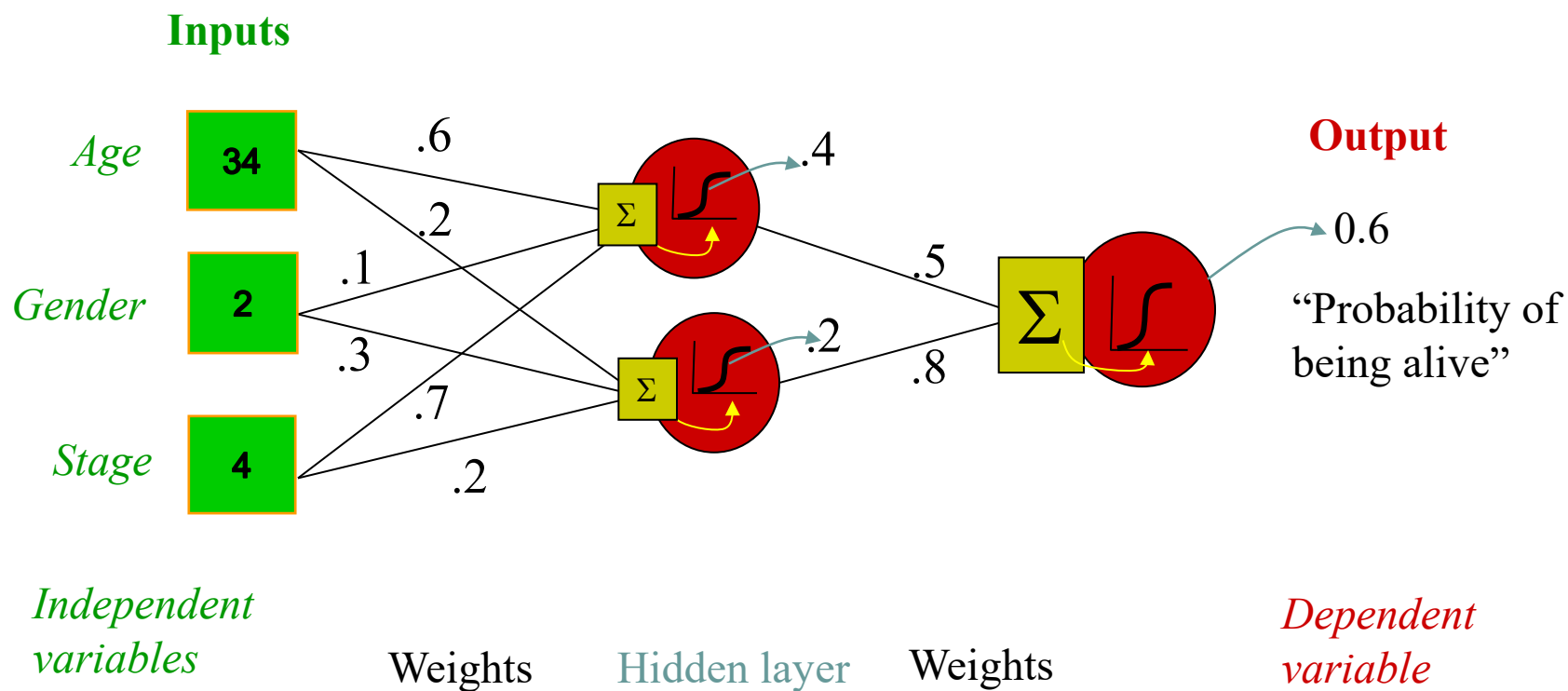
# Sigmoidal activation function defined by Lambda function
f = lambda z: 1.0/(1.0+np.exp(-z))

# Computing hidden layers
h1 = f(np.dot(W1, x))
h2 = f(np.dot(W2, x))
h = np.array([h1, h2]) # output vector from hidden layer
h = np.append(b2, h) # output becomes input to output layer

# Producing the output from the network
yhat = f(np.dot(W3, h))

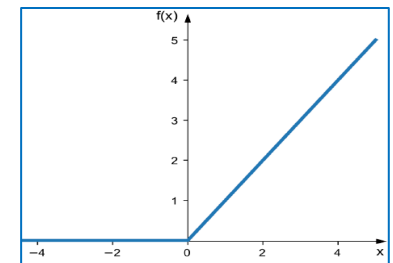
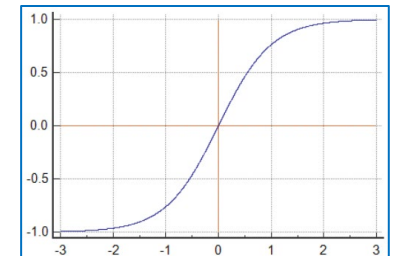
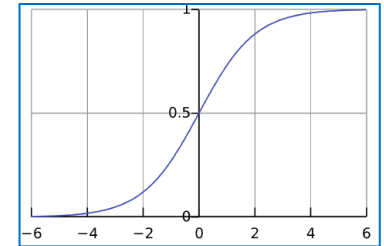
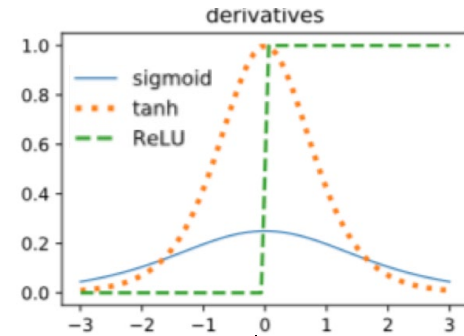
# Computing the loss
loss = np.square(y - yhat)
print("Loss: ", loss)
```

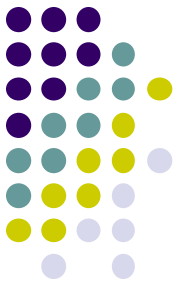
MLP with Non-linear Activation



Popular Activation Functions

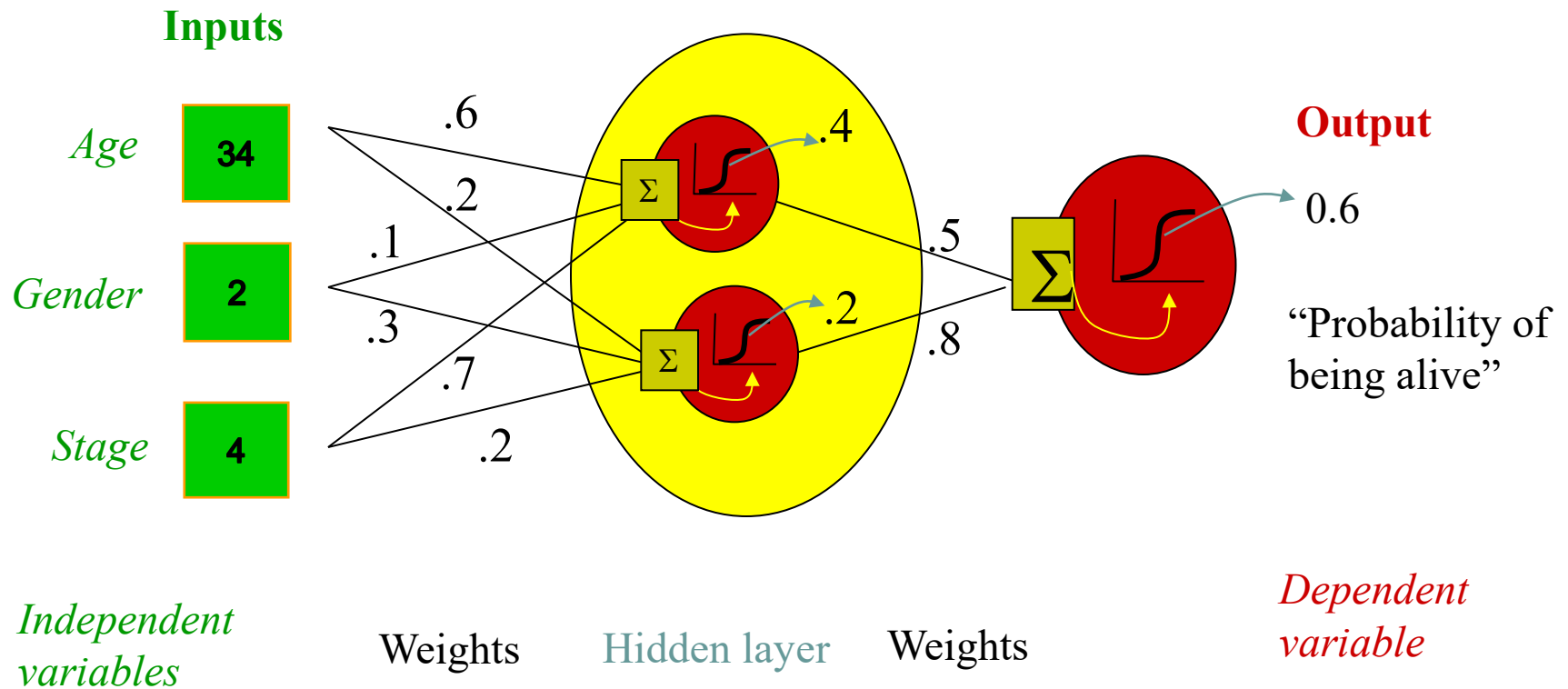
- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$
 - The output **[0,1]**, **not** widely used in modern ANNs
 - **Logistic regression** is **single neuron with sigmoid function**.
 - Variation of sigmoid:
 - Softsign(z) = $\frac{1}{1+|z|}$ (polynomial)
- **Hyperbolic tangent:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$
 - A linear transformation of sigmoid function (shifted sigmoid)
 - The output **[-1,1]**
- **ReLU** (Rectified Linear Unit): $ReLU(z) = \max(0, z)$
 - The output **[0, z]** (any value < 0 is clipped, not upper bounded)
 - Better gradient propagation
 - Fast training, not suffer from vanishing gradient – popular
 - Variations of ReLU:
 - Leaky ReLU (small slope for sparse gradient), maxout (piecewise approx.)
- **Radial Basis Function (RBF):** Gaussian function



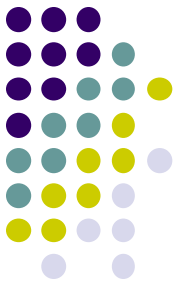


Hidden Neurons in a Network

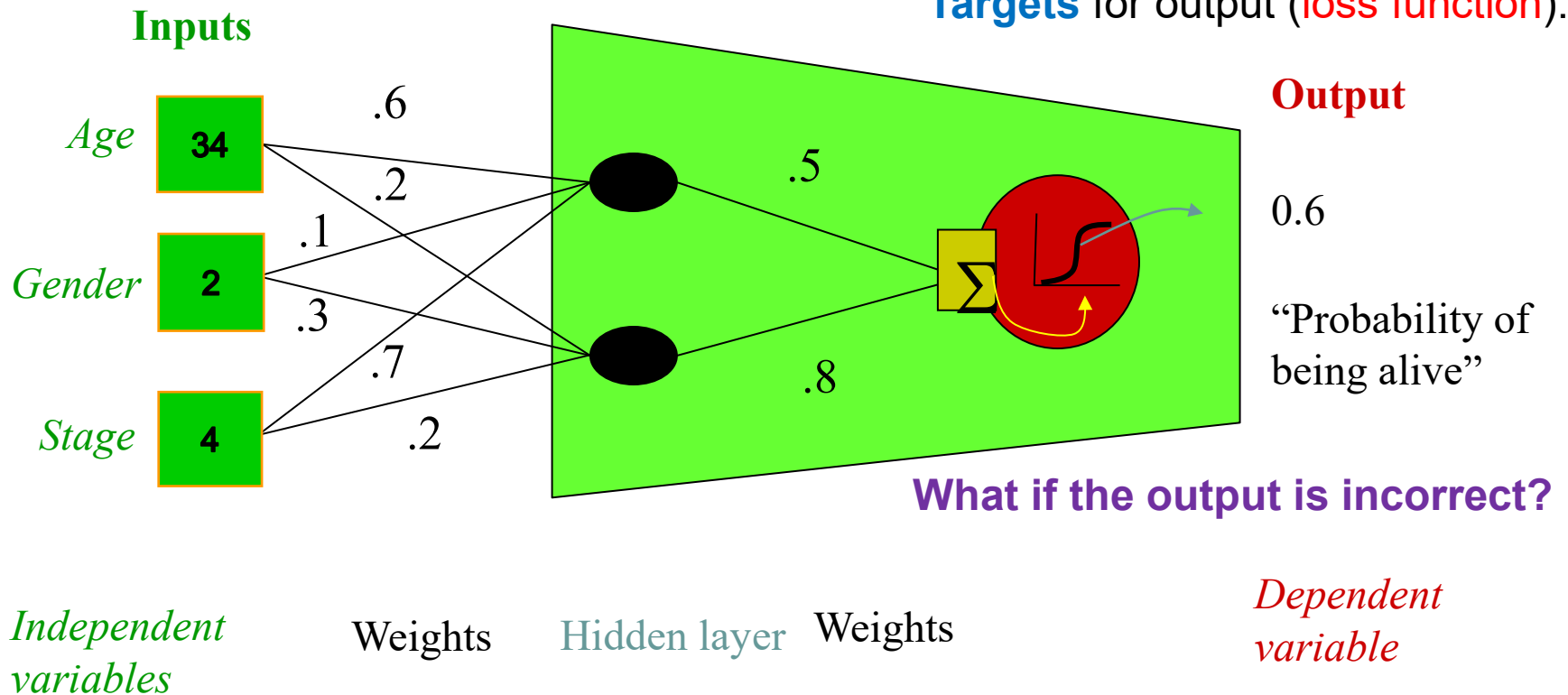
NO targets for hidden neurons (**NO loss function**)

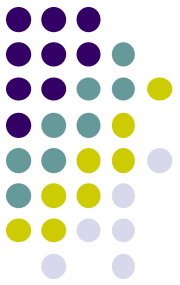


Output from a Network

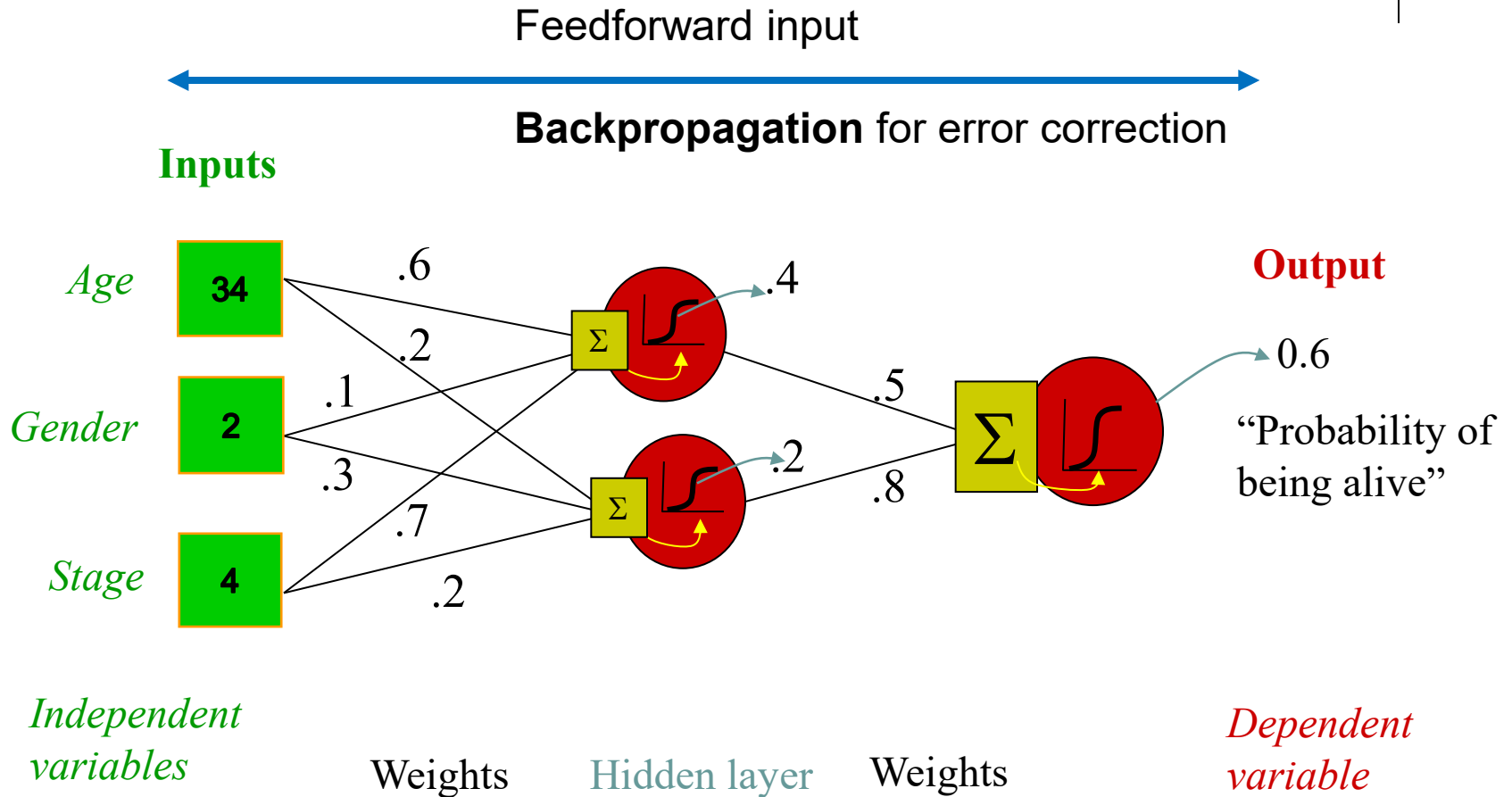


Targets for output (loss function).





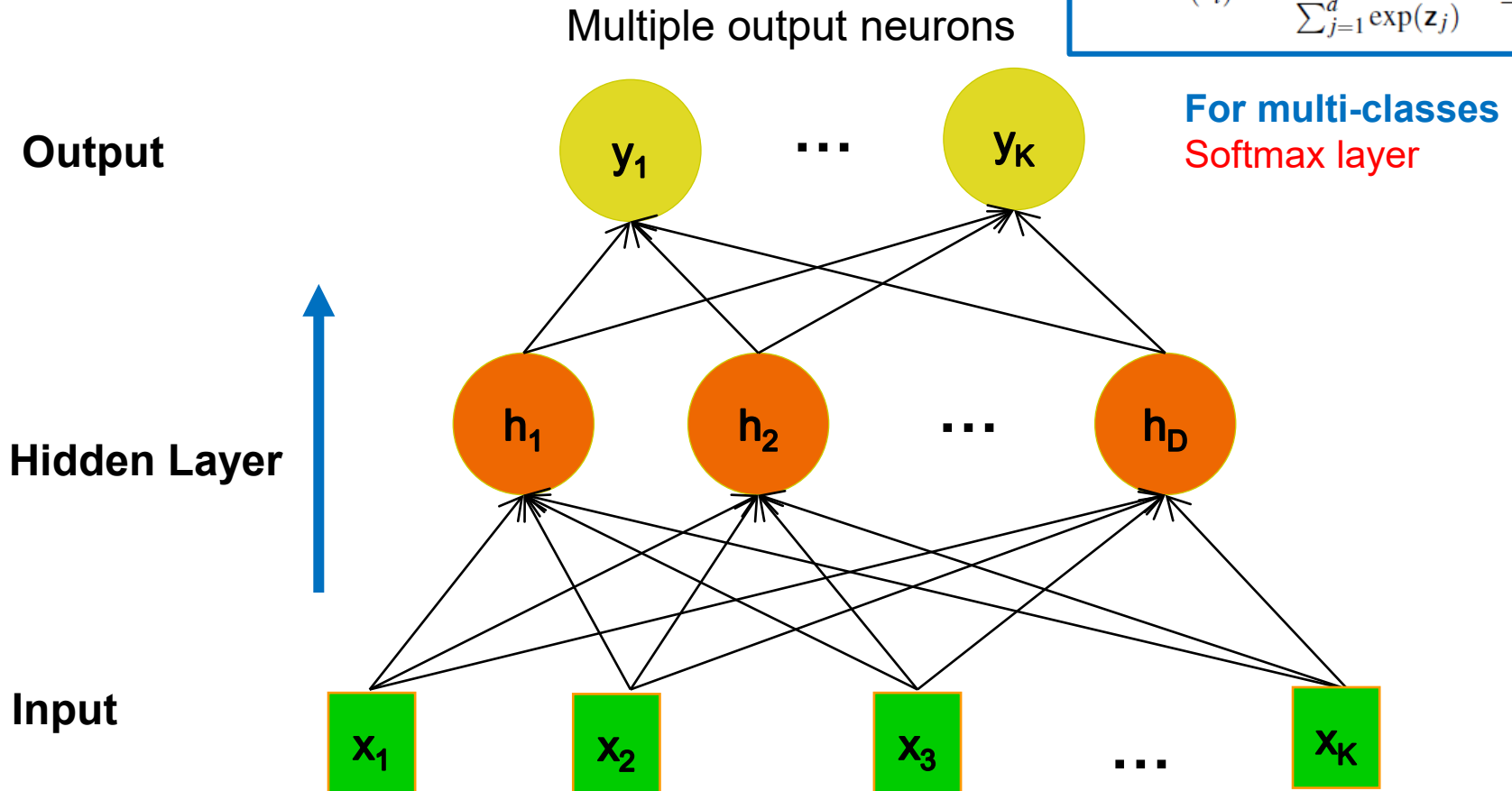
Feedforward and Backpropagation

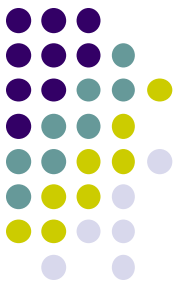


Building a Neural Network



$$\text{softmax}(\mathbf{z}_i) = \frac{\exp(\mathbf{z}_i)}{\sum_{j=1}^d \exp(\mathbf{z}_j)} \quad 1 \leq i \leq d$$





Multilayer Perceptron (MLP)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

# Load the Iris dataset
iris = load_iris()
X, y = iris.data, iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Standardize the data (important for neural networks)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

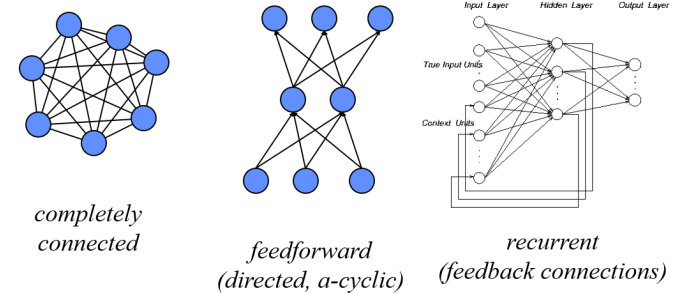
# Create and configure the neural network
mlp = MLPClassifier(hidden_layer_sizes=(10,), # Single hidden layer with 10 neurons
                    max_iter=1000,          # Maximum number of iterations
                    activation='relu',       # Activation function for hidden layer
                    solver='adam',           # Optimization algorithm
                    random_state=42)

# Train the neural network
mlp.fit(X_train, y_train)

# Predict on the test set
y_pred = mlp.predict(X_test)

# Evaluate the model
accuracy = accuracy_score(y_test, y_pred)
print("Test Accuracy:", accuracy)
```

Architectural Decisions



● Topologies of networks

- Number of hidden layers (**depth**)
- Number of neurons per hidden layer (**width**)
- The direction of flow: **forward** (input data) and **backward** (error correction)
 - Generalized to “**computational graphs**” where graphs express mathematical functions

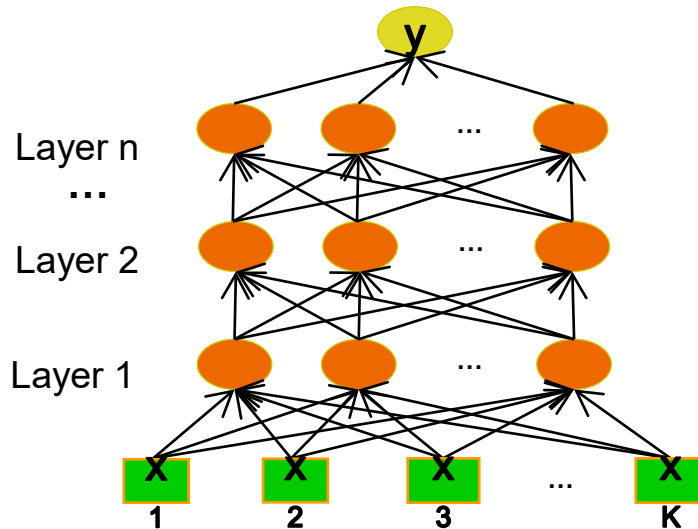
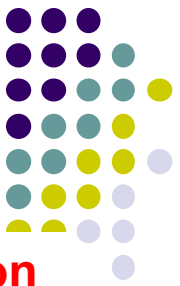
● Learning algorithms

- **Encoding schemes** (problems transformed for the algorithm)
- **Types of activation functions** (linearity/nonlinearity)
- **Forms of objective (loss) functions** (least square, cross-entropy)

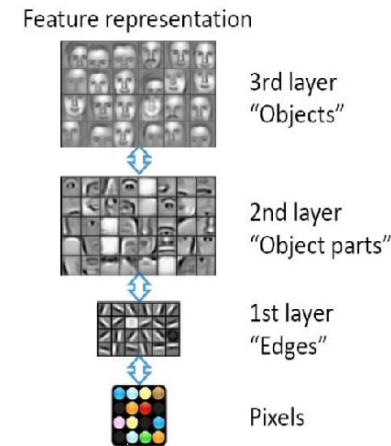
● The different architectures can solve different problems.

- Regression, classification, clustering
- Encoding, data compression, representation learning
- Sequence modeling (for Natural Language Processing)
- Generative modeling

Deep Networks for Deep Learning



Different levels of abstraction



- **Deep learning**

- Multiple levels of nonlinear processing units for feature extraction and transformation to learn a **hierarchy of concepts**
 - Like ensemble learning
- Many experimental results suggest that **deeper networks learn better**.
 - Computer vision, speech recognition, natural language processing, drug design, etc.

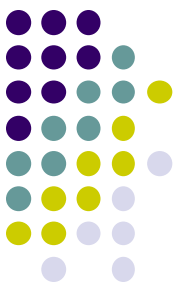
- **What is the right level of abstraction?**

- Manual design or let the model figure it out!

A Brief History of ANN



- **1943: McCulloch–Pitts** “neuron” started the field.
- **1958: Rosenblatt’s perceptron** learned its own weight values.
- **1969: Minsky & Papert** proved limitations of single-layer perceptron.
- 1970: Seppo Linnainmaa described an idea of error backpropagation.
- 1974: Werbos introduced an *idea* of multi-layer perceptron.
- **1982: Hopfield** introduced associative network and *energy function* concept
- **1985:** Boltzmann machine (probabilistic generative model (**Hinton** et al.)
- **1986:** Boom of backpropagation of errors for training multilayer networks
- **1997:** Recurrent Neural Network (**RNN**) and **LSTM** for speech recognition
- 1998: Modern image classification: LeNet for zip code recognition
- **2012:** Convolutional Neural Network (**CNN**) for image classification
 - AlexNet error ~18% vs. human error ~5.1%
- **Present: Probabilistic interpretations, Bayesian, Deep learning, etc.**
 - Since 1980s, **the form of models has not changed much** but **lots of new tricks** such as more layers, better optimization, new nonlinear functions (ReLU), faster computers (GPUs, clusters)
 - **Why is everyone talking about ANN and Deep Learning?**
 - **Alpha go** by DeepMind (**Demis Hassabis**) defeated the world champion, Lee Sedol, 2016 using **Deep Reinforcement Learning**.
 - **New York Times** and other news media made the front-page news
 - **Google acquired the company for \$400 millions**; Lots of money invested
 - **Large Language Models** like ChatGPT



Key Elements for Deep Learning

- **Tensors** (torch.tensor)

- Core data structure like Numpy arrays, but runs on GPU and autograd
 - Common operations are similar to those for ndarrays, e.g., rand, ones, zeros, indexing, slicing, reshape, transpose, element wise multiplication, matrix product
 - Tensors are created using torch.tensor().

- **Autograd** (torch.autograd)

- Tensors for computations and automatic differentiation
 - .backward() triggers gradient computation
 - .grad stores computed gradients

- **Module** (torch.nn)

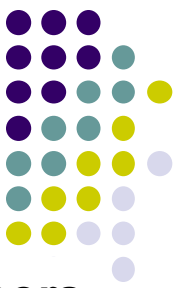
- A neural network layer that may store state or learnable weights.
 - Predefined layers: nn.Linear, nn.Conv2d, nn.ReLU, etc.
 - Loss functions: nn.CrossEntropyLoss, nn.MSELoss
 - Optimizers: torch.optim

```
# Create tensors.
x = torch.tensor(1., requires_grad=True)
w = torch.tensor(2., requires_grad=True)
b = torch.tensor(3., requires_grad=True)

# Build a computational graph.
y = w * x + b    # y = 2 * x + 3

# Compute gradients.
y.backward()

# Print out the gradients.
print(x.grad)    # x.grad = 2
print(w.grad)    # w.grad = 1
print(b.grad)    # b.grad = 1
```



Tensors on GPU

- PyTorch tensors can run on GPU.
- To run on GPU, just cast tensors to a CUDA datatype.
- To create random **tensors** for data and weights
- **Forward pass**: computing predictions and loss
- **Backward pass**: *manually* computing gradients
- **Gradient descent step** on weights

Two-layer network using tensors

```
import torch

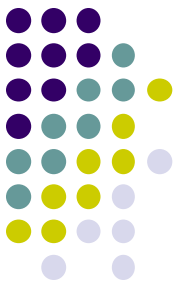
dtype = torch.cuda.FloatTensor

N, D_in, H, D_out = 64, 1000, 100, 10
x = torch.randn(N, D_in).type(dtype)
y = torch.randn(N, D_out).type(dtype)
w1 = torch.randn(D_in, H).type(dtype)
w2 = torch.randn(H, D_out).type(dtype)

learning_rate = 1e-6
for t in range(500):
    h = x.mm(w1)
    h_relu = h.clamp(min=0)
    y_pred = h_relu.mm(w2)
    loss = (y_pred - y).pow(2).sum()

    grad_y_pred = 2.0 * (y_pred - y)
    grad_w2 = h_relu.t().mm(grad_y_pred)
    grad_h_relu = grad_y_pred.mm(w2.t())
    grad_h = grad_h_relu.clone()
    grad_h[h < 0] = 0
    grad_w1 = x.t().mm(grad_h)

    w1 -= learning_rate * grad_w1
    w2 -= learning_rate * grad_w2
```

PyTorch: Module

- A neural network layer; may store state or learnable weights
 - `torch.nn` is the higher-level wrapper to work with neural networks, similar to Keras, but only one.
- Various layers: Dropout, Linear, Normalization, Convolution, Pooling

⊟ torch.nn

Parameters

⊟ Containers

⊟ Convolution Layers

Conv1d

Conv2d

Conv3d

ConvTranspose1d

ConvTranspose2d

ConvTranspose3d

⊟ torch.nn

Parameters

⊟ Containers

⊟ Convolution Layers

⊟ Pooling Layers

MaxPool1d

MaxPool2d

MaxPool3d

MaxUnpool1d

MaxUnpool2d

MaxUnpool3d

AvgPool1d

AvgPool2d

AvgPool3d

FractionalMaxPool2d

LPPool2d

AdaptiveMaxPool1d

AdaptiveMaxPool2d

AdaptiveMaxPool3d

AdaptiveAvgPool1d

AdaptiveAvgPool2d

AdaptiveAvgPool3d

⊟ Loss functions

L1Loss

MSELoss

CrossEntropyLoss

NLLLoss

PoissonNLLoss

KLDivLoss

BCELoss

BCEWithLogitsLoss

MarginRankingLoss

HingeEmbeddingLoss

MultiLabelMarginLoss

SmoothL1Loss

SoftMarginLoss

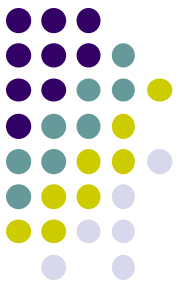
MultiLabelSoftMarginLoss

CosineEmbeddingLoss

MultiMarginLoss

TripletMarginLoss

Module allows
creating various
neural networks.



Example Neural Network using PyTorch

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # Use GPU if available

# Define the neural network architecture
class SimpleNN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super(SimpleNN, self).__init__()
        # Define the layers
        self.fc1 = nn.Linear(input_size, hidden_size) # First layer
        self.fc2 = nn.Linear(hidden_size, output_size) # Second (output) layer
        self.sigmoid = nn.Sigmoid() # Sigmoid activation for output

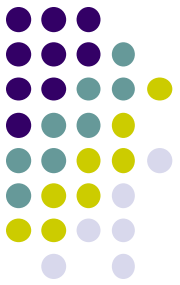
    def forward(self, x):
        x = torch.sigmoid(self.fc1(x)) # Apply sigmoid to the first layer
        x = self.sigmoid(self.fc2(x)) # Apply sigmoid to the second layer
        return x

# Generate random data (2 input features for binary classification)
np.random.seed(42)
X = np.random.randn(1000, 2).astype(np.float32) # 1000 samples, 2 features
y = (np.random.randn(1000, 1) > 0).astype(np.float32) # Binary labels (0 or 1)

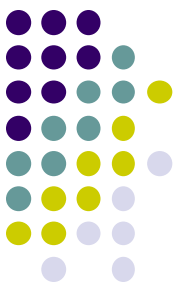
# Convert data to PyTorch tensors and move to GPU
X_tensor = torch.tensor(X).to(device)
y_tensor = torch.tensor(y).to(device)

# Initialize the model
model = SimpleNN(2, 4, 1).to(device)
```

This example is an *incomplete* implementation of a neural network model.

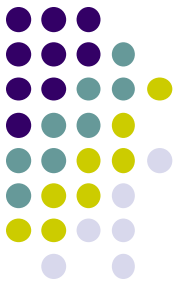


Training Artificial Neural Networks



The Recipe for Supervised Learning

1. Given a training data set, $\{(X_1, \dots, X_k, Y_j) | j = 1 \dots N\}$
2. Define the **objective function** from a decision function $\hat{y}_i = f_\theta(x_i)$ in terms of loss (cost) or **benefit**:
 - **Benefit:** $\theta^* = \operatorname{argmax}_\theta \sum_{i=1}^N g_\theta(\hat{y}_i, y_i)$ or
 - **Loss** $\theta^* = \operatorname{argmin}_\theta \sum_{i=1}^N l_\theta(\hat{y}_i, y_i)$
 - The common form of objective function is a cost function $l(\hat{y}_i, y_i) \in \mathbb{R}$.
 - Is there any analytical solution? If yes, use it, If no, use an **optimization approach** such as **Gradient Descent/Ascent**, **Newton-Rapson**, **Simulated Annealing**.
 - Define **log loss function**: $\theta^* = \operatorname{argmin}_\theta \sum_{i=1}^N J_\theta(\hat{y}_i, y_i)$
3. Learn the model θ^* using:
 - Gradient: $\nabla J(f_\theta(x_i), y)$ (by taking the derivative of the log loss)
 - Parameter update rule: $\theta^{t+1} = \theta^t - \eta \nabla J(f_\theta(x_i), y_i)$ where η is a learning rate
4. Predict $\widehat{y_{new}}$ for x_{new} using $f_{\theta^*}(x_{new})$



Gradients for the Weight Update

- **Objective function**

- Squared loss: $l(\hat{y}_i, y_i) = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$ where $\hat{y}_i = f(x; w)$
- Cross-entropy loss: $l(\hat{y}_i, y_i) = -\sum_i y_i \log \hat{y}_i$
 - $\hat{y}_i$ can be a value from Softmax function $[0,1]$ in the output layer.
- $w^* = \operatorname{argmin}_w \sum_{i=1}^N l(\hat{y}_i, y_i)$

- **Weight update with a gradient**

- Weight update rule
 - $w^{t+1} = w^t - \alpha \frac{\partial}{\partial w} l(\hat{y}_i, y_i)$

- **Derivatives** for commonly used activation functions

- $\sigma(z) = \frac{1}{1+e^{-z}}$

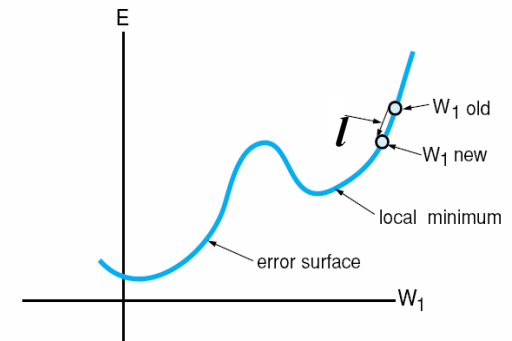
$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

$$\frac{\partial \tanh(z)}{\partial z} = \frac{1}{\cosh^2(z)}$$

- $\operatorname{ReLU}(z) = \max(0, z)$

$$\frac{\partial \operatorname{ReLU}(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

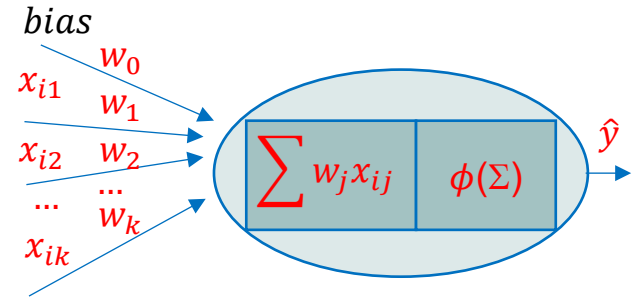


Backpropagation



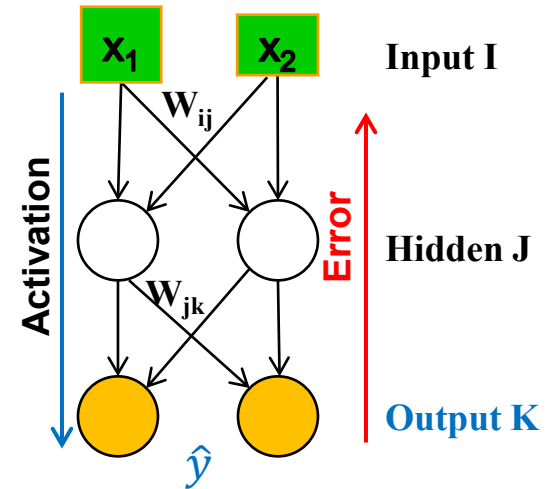
- **Backpropagation to correct errors**

- An efficient gradient-based optimization of the weights in a multilayer network



- **Algorithm:**

- Initialize w, α
- Repeat for each example x_i until convergence
 - Compute and propagate the output $\hat{y}$ (forward pass).
 - Propagate the gradients backward (backward pass) if there is any error.
 - Update each weight w (via gradient descent)



Weight Update for an Output Node



- **Error from an output node:** $E = l(\hat{y}_i, y_i) \approx \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$

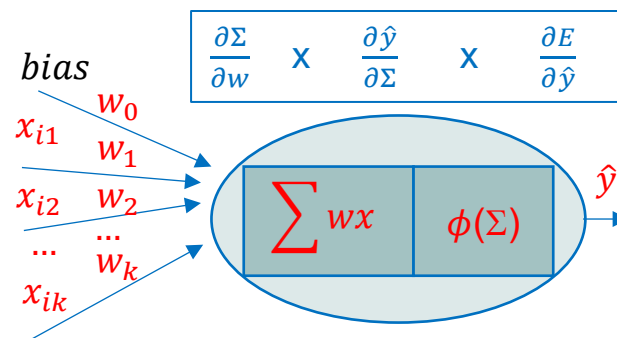
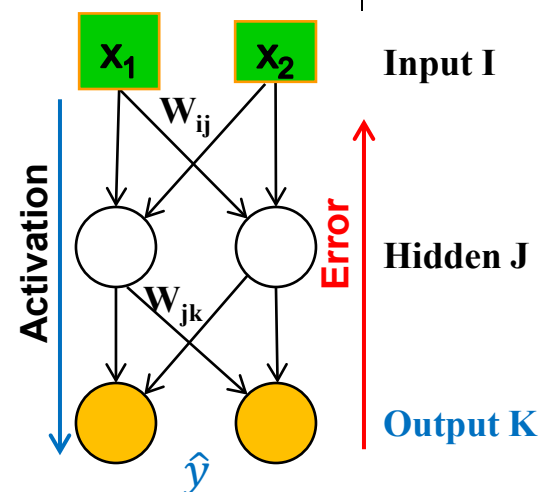
- Let $\Sigma = \Sigma w x$ and assume $\hat{y}_i = \sigma(\Sigma) = \frac{1}{1+e^{-\Sigma}}$
- We want $\Delta w \propto -\alpha \frac{\partial E}{\partial w}$. Since we **cannot** directly calculate $\frac{\partial E}{\partial w}$
 - **Use chain rule:** $y=g(u), u=h(x), \frac{\partial y}{\partial x} = \frac{\partial y}{\partial u} \times \frac{\partial u}{\partial x}$

- **Chain rule:** $\frac{\partial E}{\partial w_{jk}} = \frac{\partial \Sigma}{\partial w_{jk}} \times \frac{\partial \hat{y}}{\partial \Sigma} \times \frac{\partial E}{\partial \hat{y}}$

- $\frac{\partial \Sigma}{\partial w_{jk}} = \frac{\partial \Sigma w_{jk} \hat{o}_j}{\partial w_{jk}} = \hat{o}_j$ ($\hat{o}_j$ is input to output node k)
- $\frac{\partial \hat{y}}{\partial \Sigma} = \frac{\partial}{\partial \Sigma} \left(\frac{1}{1+e^{-\Sigma}} \right) = \hat{y}(1 - \hat{y})$ assuming **sigmoid function**
- $\frac{\partial E}{\partial \hat{y}} = \frac{1}{2} \frac{\partial (\hat{y} - y)^2}{\partial \hat{y}} = (\hat{y} - y)$
- $\frac{\partial E}{\partial w_{jk}} = \frac{\partial \Sigma}{\partial w_{jk}} \times \frac{\partial \hat{y}}{\partial \Sigma} \times \frac{\partial E}{\partial \hat{y}} = \hat{o}_j \hat{y}(1 - \hat{y})(\hat{y} - y)$

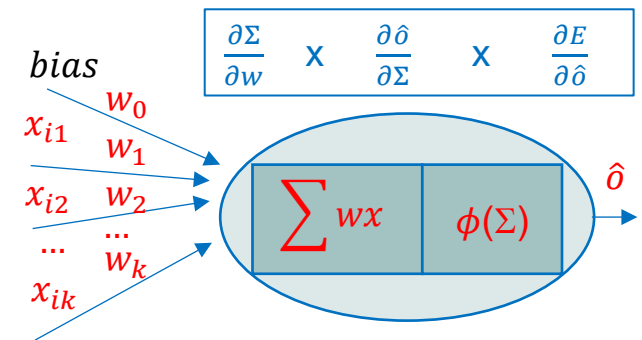
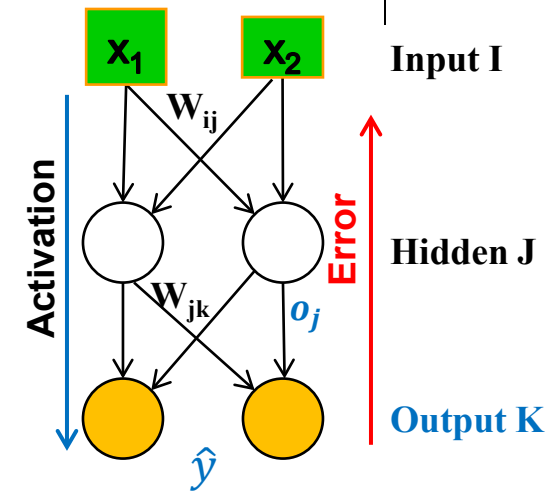
- **The weight update by gradient:**

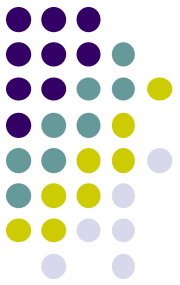
- $w^{t+1} = w^t - \alpha \frac{\partial}{\partial w} E$
- $w^{t+1} = w^t - \alpha \hat{y}(1 - \hat{y}) \hat{o}_j (\hat{y} - y)$
 - $\hat{o}_j$ is an output from a node but can also be input (like x) to next node in a network.



Weight Update for a Hidden Node

- **Error from a hidden node:** $E \approx \frac{1}{2} \sum_i (o_i - y_i)^2$
 - Let $\Sigma = \Sigma w x$, output from a hidden node $\hat{o}_j = \frac{1}{1+e^{-\Sigma}}$
 - We want $\Delta w_{ij} \propto \frac{\partial E}{\partial w_{ij}}$, but we **cannot** directly calculate $\frac{\partial E}{\partial w_{ij}}$
- **Chain rule:** $\frac{\partial E}{\partial w_{ij}} = \frac{\partial \Sigma_j}{\partial w_{ij}} \times \frac{\partial \hat{o}_j}{\partial \Sigma_j} \times \frac{\partial E}{\partial \hat{o}_j}$
 - $\frac{\partial \Sigma_j}{\partial w_{ij}} = \frac{\partial (w_{ij} o_i)}{\partial w_{ij}} = \hat{o}_i$ ($\hat{o}_i$ is the input to hidden node j, x)
 - $\frac{\partial \hat{o}_j}{\partial \Sigma_j} = \frac{\partial}{\partial \Sigma_j} \left(\frac{1}{1+e^{-\Sigma_j}} \right) = \hat{o}_j (1 - \hat{o}_j)$
 - $\frac{\partial E}{\partial \hat{o}_j} = \left[\sum_k \frac{\partial \Sigma_k}{\partial \hat{o}_j} \times \frac{\partial \hat{o}_k}{\partial \Sigma_k} \times \frac{\partial E}{\partial \hat{o}_k} \right]$ ($\hat{o}_j$ is input to the k node like x)
 - The error from hidden nodes at **J** are propagated to all the connected nodes at **K** layer.
 - $\frac{\partial \Sigma_k}{\partial \hat{o}_j} = \frac{\partial (w_{jk} \hat{o}_j)}{\partial \hat{o}_j} = w_{jk}, \quad \frac{\partial \hat{o}_k}{\partial \Sigma_k} = \hat{o}_k (1 - \hat{o}_k), \quad \frac{\partial E}{\partial \hat{o}_k} = (\hat{o}_k - y_k)$
 - $\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial \hat{o}_j} \times \frac{\partial \hat{o}_j}{\partial \Sigma_j} \times \frac{\partial \Sigma_j}{\partial w_{ij}} = \hat{o}_i \hat{o}_j (1 - \hat{o}_j) \left[\sum_k w_{jk} \hat{o}_k (1 - \hat{o}_k) (\hat{o}_k - y_k) \right]$
- **The weight update by gradient:** $w_{ij}^{t+1} = w_{ij}^t - \alpha \frac{\partial E}{\partial w_{ij}}$





Gradients for the Weight Update

- **Objective function**

- Squared loss: $l(\hat{y}_i, y_i) = \frac{1}{2} \sum_i (\hat{y}_i - y_i)^2$ where $\hat{y}_i = f(x; w)$
- Cross-entropy loss: $l(\hat{y}_i, y_i) = -\sum_i y_i \log \hat{y}_i$
 - $\hat{y}_i$ can be a value from Softmax function $[0,1]$ in the output layer.
- $w^* = \operatorname{argmin}_w \sum_{i=1}^N l(\hat{y}_i, y_i)$

- **Weight update with a gradient**

- Weight update rule
 - $w^{t+1} = w^t - \eta \frac{\partial}{\partial w} l(\hat{y}_i, y_i)$

- **Derivatives** for commonly used activation functions

- $\sigma(z) = \frac{1}{1+e^{-z}}$

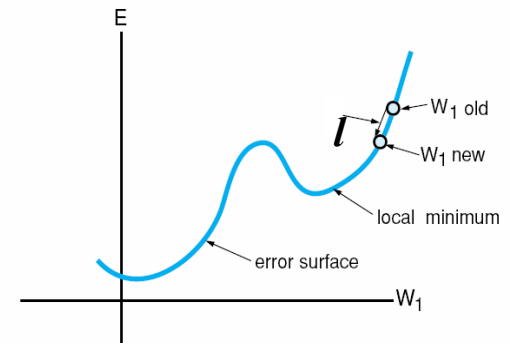
$$\frac{\partial \sigma(z)}{\partial z} = \sigma(z)(1 - \sigma(z))$$

- $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$

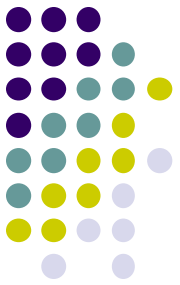
$$\frac{\partial \tanh(z)}{\partial z} = \frac{1}{\cosh^2(z)}$$

- $\operatorname{ReLU}(z) = \max(0, z)$

$$\frac{\partial \operatorname{ReLU}(z)}{\partial z} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

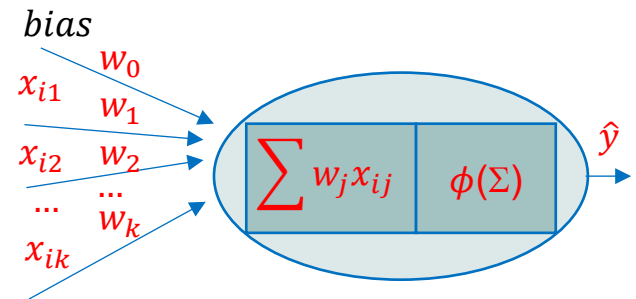


Backpropagation in Neural Networks



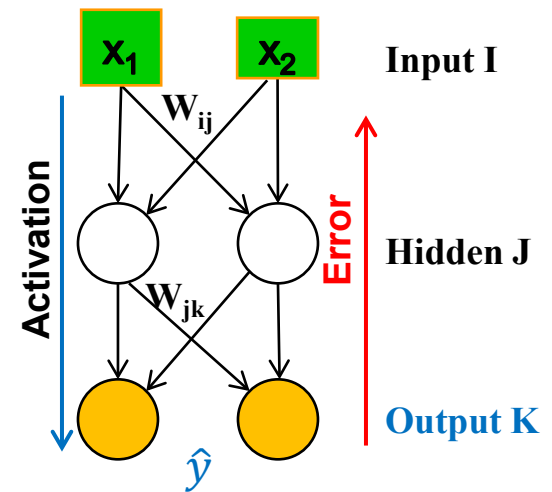
- **Backpropagation to correct errors**

- An efficient gradient-based optimization of the weights in a multilayer network

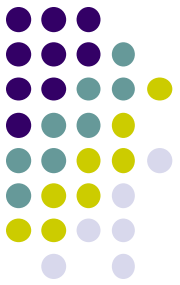


- **Algorithm:**

- Initialize w , η
- Repeat for each example x_i until convergence
 - Compute and propagate the output $\hat{y}$ (**forward pass**).
 - Propagate the gradients backward (**backward pass**) if there is any error.
 - Update each weight w (via gradient descent)



Backpropagation Algorithm



- Update the weight w for a node: $w^{t+1} = w^t - \eta \Delta w$
- Compute Δw :

Every node computes $\hat{y}(1 - \hat{y})x$ from $\frac{\partial \hat{y}}{\partial \Sigma} \times \frac{\partial \Sigma}{\partial w_{jk}}$ (based on the *sigmoid function*)

plus, each output node computes the error term $(\hat{y} - y)$ from $\frac{\partial E}{\partial \hat{y}}$

plus, each hidden node j computes for all K connected nodes: $\sum_k \delta_k w_{jk}$ where the error term for node k : $\delta_k = o_k(1 - o_k)(o_k - y_k)$ from $\sum_k \frac{\partial o_k}{\partial \Sigma_k} \times \frac{\partial E}{\partial o_k} \times \frac{\partial \Sigma_k}{\partial o_j} = \sum_k \delta_k w_{jk}$

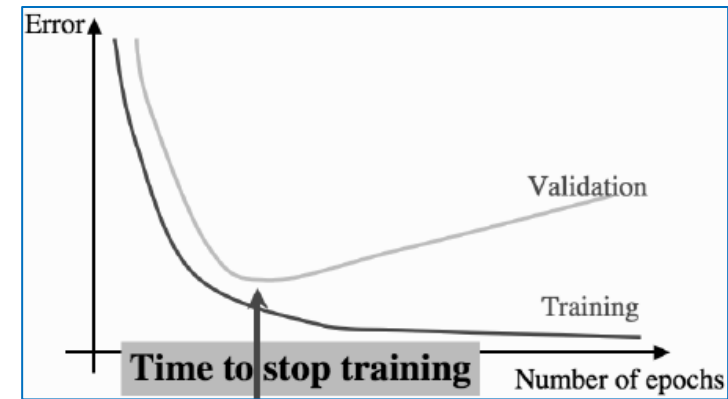
- For each output node $\Delta w = \hat{y}(1 - \hat{y})x(\hat{y} - y)$
- For each hidden node j for all k nodes $\Delta w = \hat{y}(1 - \hat{y})x \sum_k [o_k(1 - o_k)(o_k - y_k)w_{jk}]$
- Backpropagation is just a repeated application of the chain rule.
- **Algorithm**
 - Instantiate the computation as a **directed acyclic graph**, where each **intermediate quantity** will be stored at a **node**:
 - (a) the quantity computed in the forward pass and
 - (b) the partial derivative of the goal with respect to that node's intermediate quantity.
 - Initialize all partial derivatives to 0.
 - Visit each node in **reverse topological order** and add its contribution to the partial derivatives of its parents.

Steps for Training Neural Networks



1. **Preprocess** the training data.
2. **Split the data** into **training**, **validation**, and **testing sets**.
 - The **validation set** is to validate how well the network is learning during training.
3. **Design a network architecture**.
 - The more complex the network, the longer it will take for training, can be subject to overfitting.
 - Try several architectures with different numbers of hidden nodes and layers, starting with a smaller # of nodes, then gradually increase the nodes.
 - Select the appropriate hyperparameters.
4. **Train the network**.
 - After a few iterations of the algorithm through the entire training data (1 epoch), the generalization ability should be tested using the **validation set**.
 - Stop the training when the error on the validation set starts to increase.
5. **Test the network**.
 - Test the network with the **test data set** to measure the performance.
6. **Repeat steps 3 – 5** changing hyperparameters **until** converged or satisfied.

When to Stop Training

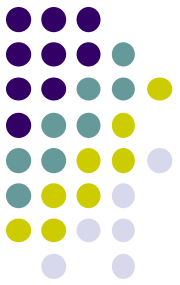


- **Criteria to stop training**

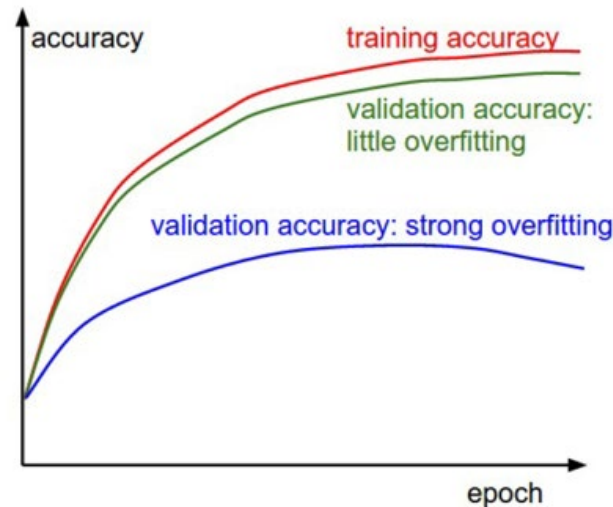
- Predefined # of iterations – may not learn enough
- Until converged – risk of overfitting
- Until a predefined minimum error is reached – may never terminate or overfit

- **Early stopping**

1. **Use a validation set** (different from the testing data)
2. **Monitor** the generalization ability **during training** by plotting the errors (e.g., MSE)
3. **Find the right stopping point**



Overfitting in Neural Networks



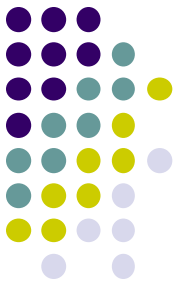
- **Dropout**

- Dropping out some neurons (regularization), commonly used in deep learning

- **Empirical risk minimization by regularization**

- $\operatorname{argmin}_w \frac{1}{N} \sum l(\hat{y}, y) + \lambda R(w)$ where $R(w)$ is a regularized term.
 - L1 regularization term: $\|w\|_1$
 - L2 regularization term: $\|w\|_2^2$

Strengths and Weaknesses of ANN



- **Strengths**

- Can handle high-dimensional data for both classification and regression
- Quite robust to noise data or data with errors (**fault tolerance**)
- Can be highly parallel and distributed (**parallel and distributed computing**)
- Distributed representation and storage of data (**computational graph**)
- Can learn when the form of target function is unknown (**non-parametric**)
- Possible for adaptive learning (**online learning**)
- **End-to-end learning**

- **Weaknesses**

- Sensitive to initial weights and hyperparameters
- Long training time and may not find the optimal weights (**high computing resource needed**)
- Difficult to explain why a target function is returned (**difficult to interpret**)
- Not easy to design the architecture to solve complex problems